

Weiterentwicklung eines selbstfahrenden Fahrzeugs mit Lidar und anderen Sensoren

STUDIENARBEIT

des Studienganges Informatik
an der Dualen Hochschule Baden-Württemberg Ravensburg Campus Friedrichshafen

von

Elena Schwarzbach, 3830156, TIK23

Sonia Sinaci, 8366601, TIK23

Scott Jonathan Hebach, 8958697, TIT23

Marius Maurer, 9339665, TIT23

13. Juli 2026

Bearbeitungszeitraum: 28 Wochen

Gutachter der Dualen Hochschule: Titel Claudia Zinser

Eidesstattliche Erklärung

gemäß Ziffer 1.1.13 der Anlage 1 zu §§ 3, 4 und 5 der Studien- und Prüfungsordnung für die Bachelorstudiengänge im Studienbereich Technik der Dualen Hochschule Baden-Württemberg vom 29.09.2017.

Wir versichern hiermit, dass unsere Bachelorarbeit (bzw. Projektarbeit oder Studienarbeit bzw. Hausarbeit) mit dem Thema:

Weiterentwicklung eines selbstfahrenden Fahrzeugs mit Lidar und anderen Sensoren

selbstständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt habe. Wir versichern zudem, dass die eingereichte elektronische Fassung mit der gedruckten Fassung übereinstimmt.

Friedrichshafen, 13. Juli 2026



Elena Schwarzbach



Sonia Sinaci



Scott Jonathan Hebach

Marius Maurer

Genderhinweis

In der vorliegenden Arbeit wird aus Gründen der sprachlichen Ökonomie und zur Sicherung eines einheitlichen Schriftbildes auf eine durchgängige geschlechts-spezifische Differenzierung verzichtet. Sämtliche Personen-bezeichnungen im generischen Maskulinum verstehen sich als geschlechtsneutral und schließen alle Geschlechter gleichermaßen ein. Diese Entscheidung basiert rein auf redaktionellen Überlegungen und spiegelt keine Geringschätzung gegenüber weiblichen oder nicht-binären Personen wider. Die Autoren sind sich der Wirkung von Sprache auf die Wahrnehmung bewusst und verfolgen mit dieser Wahl das Ziel, die Komplexität des Textes zugunsten der inhaltlichen Klarheit zu reduzieren.

Inhaltsverzeichnis

Abkürzungsverzeichnis	I
Abbildungsverzeichnis	IV
Tabellenverzeichnis	V
1 Einleitung	1
2 Ist-Zustand	2
2.1 App-Steuerung	2
2.2 Bestehende funktionale Komponenten	2
2.3 Probleme	3
3 Technikkorridor	5
3.1 Hardwarevergleich Raspberry Pi 4 und Raspberry Pi 5	5
3.2 Vergleich unterschiedlicher Betriebssysteme für Raspberry Pi	7
3.3 Softwarearchitektur	9
3.4 Betrachtete Systemarchitekturen	10
3.5 Kommunikationskonzept	11
3.6 Entscheidung und Begründung	12
3.7 Fazit	13
4 Methodik	16
4.1 Nutzwertanalyse	16
5 Experimentelles Evaluationsdesign	18
5.1 Methoden zur Hindernisumfahrung	18
5.1.1 Mathematische Formeln für die Berechnung der Fahrbahn	18
5.1.2 Berechnung Lenkwinkel	21
6 Autoumbau	23
6.1 Umverteilung des Gewichtes	23
6.1.1 Hochkante Einlage in den Mittelteil	23
6.1.2 Frankenstein Umbau	24

6.1.3	Einkaufsrاد Lösung	25
6.1.4	Nutzwertanalyse zur Gewichtsumverteilung	25
7	Containerisierung und Systemstart	26
7.1	Ausgangslage und Probleme	26
7.2	Multi-Stage-Build	27
7.3	Deterministischer Systemstart	28
7.4	Plattformübergreifende Lauffähigkeit	29
8	Implementierung Autonomes Fahren	30
8.1	Datenverarbeitung	30
8.1.1	Wechsel von der Ultraschallsensorik auf den LiDAR	30
8.1.2	Architektur der Verarbeitungskette	31
8.1.3	Treiberknoten und Aufbereitung der Scandaten	32
8.1.4	Stabilisierung der seriellen LiDAR-Kommunikation	33
8.1.5	Sicherheitsgerichtete Auswertung: Not-Halt-Funktion	35
8.1.6	Heatmap-basierte Visualisierung der LiDAR-Daten	36
8.1.7	Verarbeitung der Laufdaten	36
8.2	Hinderniserkennung	38
8.3	Gauß-basierte Hindernisvermeidung	39
8.4	Optimierung der Spurplanung	41
8.5	Priorisierung der Fahrbefehle	42
9	Aktualisierung der Applikation	44
10	Ergebnisse	46
10.1	Qualitative Bewertung	46
10.2	Herausforderungen und Weiterentwicklungen	47
11	Diskussion	49
12	Fazit und Ausblick	50
12.1	Ausblick und Weiterentwicklungsmöglichkeiten	50
12.1.1	Weiterentwicklung der LiDAR-Datenverarbeitung	51
12.1.2	Live-Heatmaps und Umgebungskarten	52

12.1.3 Selbstlokalisierung und Kartierung (SLAM)	53
12.1.4 Erweiterung der Sicherheitsfunktion	53
12.1.5 Systematisierung der Laufdatenverarbeitung	54
12.1.6 Automatisierung der Container-Infrastruktur	55
12.1.7 Zusammenfassende Einordnung	55
Literaturverzeichnis	V
Anhang	VII
A Anhang	VII

Abkürzungsverzeichnis

A Ampere.

A* A*.

API Application Programming Interface.

ARM64 Advanced RISC Machine 64-bit.

Baudrate Baudrate.

Bresenham Bresenham-Algorithmus.

C++ C++.

CI/CD Continuous Integration/Continuous Deployment.

CPU Central Processing Unit.

CRLF Carriage Return Line Feed.

DataFrame DataFrame.

DBSCAN Density-Based Spatial Clustering of Applications with Noise.

DDS Data Distribution Service.

DevOps Development Operations.

Docker Docker.

Ethernet Ethernet.

Foxglove Foxglove Studio.

Gazebo Gazebo.

GB Gigabyte.

Geekbench Geekbench.

Git Git.

GitHub Actions GitHub Actions.

GPIO General Purpose Input/Output.

HDMI High-Definition Multimedia Interface.

I/O Input/Output.

IoT Internet of Things.

JSON JavaScript Object Notation.

KI Künstliche Intelligenz.

LiDAR Light Detection and Ranging.

LPDDR4 Low-Power Double Data Rate 4.

LPDDR4X Low-Power Double Data Rate 4X.

mA Milliampere.

NVMe Non-Volatile Memory Express.

OS Operating System.

PCIe Peripheral Component Interconnect Express.

Python Python.

RAM Random Access Memory.

ROS 2 Robot Operating System 2.

rosbag2 rosbag2.

RPi Raspberry Pi.

RPLIDAR RPLIDAR.

RTC Real-Time Clock.

RViz ROS Visualization.

SD Secure Digital.

SLAM Simultaneous Localization and Mapping.

SPI Serial Peripheral Interface.

SSD Solid State Drive.

TF Transform.

Unix Unix.

USB Universal Serial Bus.

USB-PD USB Power Delivery.

V Volt.

Abbildungsverzeichnis

1	Parabel Funktion	19
2	Exponential Funktion	20
3	Gaussche Glocke Funktion	20
4	Zweistufiger Multi-Stage-Build der Container-Images	28
5	Komponenten und Datenflüsse der ROS 2 Verarbeitungskette	32
6	Synchronisierter Verbindungsaufbau des LiDAR-Treibers	34
7	Zustandsverhalten der Not-Halt-Funktion	35
8	Zustandsdiagramm: Ausweichen	41
9	Klassendiagramm: Ausweichen	43

Tabellenverzeichnis

1	Direkter Hardwarevergleich zwischen Raspberry Pi 4 und Raspberry Pi 5	6
2	Vergleich relevanter Betriebssystemoptionen	8
3	Vergleich zwischen nativer ROS2-Installation und Docker-basierter ROS2-Architektur	10
4	Bewertung betrachteter Systemarchitekturen	11
5	Beispiel Nutzwertanalyse	16
6	Nutzwertanalyse zur Gewichtsverteilung	26
7	Statistische Kennwerte der aufgezeichneten LiDAR-Scans	37
8	Abgestuftes Protokollierungskonzept der Knoten	37
9	Evaluation der Hinderniserkennung und Ausweichfunktion	46
10	Weiterentwicklungspfade im Überblick	50

1 Einleitung

Die fortschreitende Entwicklung autonomer Systeme prägt zunehmend technologische Innovationen in Industrie, Logistik und Mobilität. Insbesondere im Bereich der autonomen Fahrzeuge ermöglichen sie eine Effizienzsteigerung, erhöhte Sicherheit und die Erschließung neuer Anwendungsszenarien. Die Realisierung solcher Systeme auf ressourcenbeschränkter Hardware, wie sie in Modellfahrzeugen oder spezialisierten Robotern zum Einsatz kommt, stellt jedoch eine besondere Herausforderung dar. Sie erfordert eine sorgfältige Abstimmung von Sensorik, Aktorik, Softwarearchitektur und Regelungsalgorithmen, um echtzeitnahe und robuste Funktionalität zu gewährleisten.

Diese Studienarbeit widmet sich der Konzeption und Implementierung eines teilautonomen Fahrsystems auf Basis eines existierenden Modellfahrzeugs. Im Zentrum steht die Entwicklung einer lokalen Hindernisvermeidung unter Verwendung von Light Detection and Ranging (LiDAR)-Sensorik und dem Robot Operating System 2 (ROS 2). Die wissenschaftliche Problemstellung ergibt sich aus der Notwendigkeit, komplexe Wahrnehmungs- und Planungsalgorithmen so zu adaptieren, dass sie auf einer Embedded-Plattform wie dem Raspberry Pi performant ausgeführt werden können. Ein zentraler Untersuchungsgegenstand ist dabei die Eignung eines reaktiven, auf Gaußfunktionen basierenden Ansatzes zur Trajektorienplanung, der ohne globale Kartierung eine flüssige und sichere Umfahrung statischer Hindernisse ermöglichen soll.

Das Hauptziel dieser Arbeit ist die Weiterentwicklung des Modellfahrzeugs zu einem robusten, teilautonomen System, das sowohl manuell via Smartphone-Applikation als auch autonom betrieben werden kann. Die Funktionalität wird anhand messbarer Kriterien evaluiert, darunter die zuverlässige Erkennung von Hindernissen, die Erfolgsquote von Ausweichmanövern bei definierter Geschwindigkeit und die Schaffung einer reproduzierbaren Softwareumgebung mittels Containerisierung. Die Arbeit leistet damit einen Beitrag zur praktischen Anwendung und Bewertung von Algorithmen für das autonome Fahren im Kontext ressourcenlimitierter Systeme.

2 Ist-Zustand

2.1 App-Steuerung

Die App besitzt bereits eine theoretische Steuerung über die Bewegungssensoren des Smartphones. Dafür wird in `gyroscope_data.dart` die Bibliothek `motion_sensors` verwendet. Das Widget `getGyroscope` liest bereits die Werte für Pitch, Roll und Yaw aus. Sobald neue Sensordaten eintreffen, werden Pitch und Roll auf den Bereich von -1.0 bis 1.0 begrenzt. Anschließend werden diese Werte direkt als Steuerbefehle über den `WebSocketManager` an das Fahrzeug gesendet. Dabei soll der Roll-Wert die Geschwindigkeit steuern und der Pitch-Wert für den Lenkwinkel verwendet werden. Der Yaw-Wert wird für die Steuerung nicht verwendet. Zusätzlich zeigt die App die aktuellen Sensorwerte für Pitch, Roll und Yaw auf der Oberfläche an.

In diesem Stand ist die Funktion zwar im Code angelegt, funktioniert praktisch aber noch nicht. Ein zentrales Problem liegt darin, dass die App von `motion_sensors.orientation` abhängig ist. Das Paket `motion_sensors` ist jedoch veraltet und liefert keine Werte. Außerdem werden die Sensorwerte direkt weitergegeben, ohne sie vorher zu filtern oder zu glätten. Dadurch könnten bereits kleinste Bewegungen oder Messungenauigkeiten die Steuerung beeinflussen.

Weitere Probleme sind, dass es keine Deadzone für unbeabsichtigte kleine Bewegungen gibt und keine Möglichkeit vorgesehen ist, die Empfindlichkeit der Steuerung anzupassen. Auch eine einfache Korrektur falsch herum reagierender Achsen ist nicht vorbereitet.

2.2 Bestehende funktionale Komponenten

Zu Beginn der Studienarbeit existierte bereits ein teilweise funktionsfähiger Prototyp des Modellfahrzeugs. Die grundlegende Fahrzeugplattform war aufgebaut und mit einem Raspberry Pi 4 als zentralem Steuerrechner ausgestattet. Weiterhin war bereits ein LiDAR-Sensor mechanisch am Fahrzeug montiert und elektrisch integriert. Neben der Hardware lag bereits ein erstes Architekturkonzept für die Umsetzung autonomer Fahrfunktionen vor. Dieses definierte grundlegende Komponenten für Sensorik, Fahrzeugsteuerung und Kommunikation zwischen den Softwaremodulen. Die Architektur befand sich jedoch noch auf konzeptioneller Ebene und war nicht vollständig implementiert.

Darüber hinaus existierte bereits eine Smartphone-Applikation auf Basis des Flutter-Frameworks. Die Anwendung ermöglichte die Verbindung zum Fahrzeug und bildete die Grundlage für dessen manuelle Steuerung. Funktionen für autonomes Fahren, Hinderniserkennung oder automatische Fahrentscheidungen waren zu diesem Zeitpunkt jedoch noch nicht vorhanden. Im Softwarebereich bestanden außerdem erste Ansätze für die Verarbeitung von LiDAR-Daten. Diese Komponenten arbeiteten jedoch ausschließlich mit statischen Test- und Beispieldaten und waren nicht mit der realen Sensorhardware gekoppelt. Eine operative Nutzung des LiDARs im Fahrbetrieb war daher noch nicht möglich. Zusammenfassend stand zu Beginn der Arbeit bereits eine technische Basis aus Fahrzeug, Sensorik, Softwarearchitektur und mobiler Anwendung zur Verfügung. Die vorhandenen Komponenten waren jedoch nur teilweise funktionsfähig und boten noch keine autonome Fahrfunktion.

2.3 Probleme

Aus Sicht der Sensorik und der Systeminfrastruktur bestanden zum Zeitpunkt der Ist-Analyse vier wesentliche, im Projektverlauf dokumentierte Probleme.

Erstens wurden die LiDAR-Daten nicht richtig verarbeitet. Eine operative Nutzung realer LiDAR-Hardware war nicht implementiert: Die vorhandenen LiDAR-bezogenen Komponenten arbeiteten ausschließlich auf statischen, vorab aufgezeichneten Beispieldaten, und es fehlten Treiber- und Kommunikationsschnittstellen zur Anbindung des physischen Sensors. Im Fahrbetrieb lieferte der Sensor damit keinerlei verwertbare Daten. Zweitens funktionierten alle Sensoren bis auf einen nicht. Nach der Aktualisierung des Operating System (OS) auf dem Raspberry Pi brach die gesamte General Purpose Input/Output (GPIO)-Ansteuerung, wodurch neben Motorsteuerung und Lenkung auch die Sensorik ausfiel. Der Ultraschallsensor lieferte zusätzlich aufgrund eines Konflikts mit dem Serial Peripheral Interface (SPI)-Subsystem keine gültigen Messwerte. Zeitweise stand somit nur ein einziger funktionsfähiger Sensor zur Verfügung. Die Analyse und Behebung dieser Ausfälle sowie der daraus abgeleitete Wechsel auf den LiDAR als primären Sensor werden in Abschnitt 8.1 beschrieben.

Drittens beeinträchtigte der zu große Container die Anwendung. Das damalige Container-Image enthielt neben den Laufzeitabhängigkeiten auch vollständige Entwicklungswerkzeuge und grafische Anwendungen. Auf dem ressourcenbeschränkten Raspberry Pi

führte dies zu langen Bau- und Ladezeiten, und die Anwendung funktionierte dadurch nicht richtig (Lösung siehe Kapitel 7).

Viertens war der Systemstart nicht intuitiv. Das Starten des Systems erforderte eine Abfolge manueller Schritte im Container, die erst verstanden und eingerichtet werden musste. Ohne dieses Vorwissen war das System nicht in Betrieb zu nehmen. Zugleich war die Schrittfolge fehleranfällig und führte zu schwer reproduzierbaren Zuständen (Lösung siehe Abschnitt 7.3).

3 Technikkorridor

Der Technikkorridor beschreibt den technisch sinnvollen Lösungsraum für die Realisierung eines autonomen Fahrzeugs auf Raspberry-Pi-Basis. Ziel ist es, aus den verfügbaren Hardware-, Betriebssystem- und Softwareoptionen eine Plattform abzuleiten, die für echtzeitfähige Sensorverarbeitung, Kamerabildverarbeitung, Batteriebetrieb, Wartbarkeit und Erweiterbarkeit geeignet ist.

Im Mittelpunkt steht dabei die Frage, welche Kombination aus Einplatinencomputer, Betriebssystem, Robotik-Middleware und optionaler Containerisierung für ein Studien- oder Forschungsprojekt den besten Kompromiss aus Leistungsfähigkeit, Ressourcenbedarf und Entwicklungsaufwand bietet. Für autonome Fahrzeuge sind insbesondere parallele Sensordatenströme, Bildverarbeitung, stabile Kommunikation zwischen Softwarekomponenten sowie eine robuste und wartbare Systemstruktur relevant.

Der Raspberry Pi 4 und der Raspberry Pi 5 stellen dabei zwei naheliegende Hardwareoptionen dar. Während der Raspberry Pi 4 weiterhin für bestehende oder kostenkritische Anwendungen geeignet ist, bietet der Raspberry Pi 5 durch seine höhere CPU-Leistung, bessere I/O-Anbindung und Erweiterbarkeit über PCIe eine deutlich leistungsfähigere Grundlage für Robotik- und autonome Fahrzeugsysteme [17, Abschnitte “Memory”, “Connectivity” und “Video & Sound”] [18, Abschnitte “Overview” und “Specification”]. Ergänzend beeinflussen Betriebssystemwahl, Softwarearchitektur und Kommunikationskonzept maßgeblich die Praxistauglichkeit des Gesamtsystems.

3.1 Hardwarevergleich Raspberry Pi 4 und Raspberry Pi 5

Der Hardwarevergleich dient dazu, die Eignung des Raspberry Pi 4 und des Raspberry Pi 5 für ein autonomes Fahrzeug einzuordnen. Beide Plattformen besitzen eine ähnliche Grundidee als kompakte Einplatinencomputer, unterscheiden sich jedoch deutlich hinsichtlich Rechenleistung, Speicheranbindung, I/O-Architektur und Erweiterbarkeit.

Der Raspberry Pi 4 basiert auf einer Cortex-A72-Plattform und ist mit bis zu 8 Gigabyte (GB) LPDDR4-Arbeitsspeicher verfügbar. Er bietet zwei Universal Serial Bus (USB)-3.0- und zwei USB-2.0-Anschlüsse, Gigabit-Ethernet sowie zwei Micro-High-Definition Multimedia Interface (HDMI)-Ausgänge für Dual-Display-Betrieb bis 4K. Damit ist er für allgemeine Internet of Things (IoT)-, Embedded- und Server-Anwendungen

weiterhin brauchbar. Für rechenintensive Aufgaben wie parallele Sensorverarbeitung, Bildauswertung oder mehrere gleichzeitig laufende Robotik-Komponenten stößt er jedoch früher an Leistungsgrenzen [17, Abschnitte “Memory”, “Connectivity” und “Video & Sound”].

Der Raspberry Pi 5 stellt demgegenüber die technisch modernere Plattform dar. Er verwendet eine Cortex-A76-basierte Central Processing Unit (CPU) und erreicht nach Herstellerangaben eine etwa zwei- bis dreifache CPU-Leistungssteigerung gegenüber dem Raspberry Pi 4 [18, Abschnitte “Overview” und “Specification”]. Hinzu kommen LPDDR4X-Arbeitsspeicher, der neue RP1-Input/Output (I/O)-Controller, eine Peripheral Component Interconnect Express (PCIe)-2.0-x1-Schnittstelle, eine integrierte Real-Time Clock (RTC) sowie ein Power-Button auf dem Board [18, Abschnitte “Overview” und “Specification”]. Besonders relevant ist die PCIe-Schnittstelle, da sie über entsprechende Adapter den Einsatz von Non-Volatile Memory Express (NVMe)-Solid State Drive (SSD)s ermöglicht. Dadurch können Logdaten, Kartenmaterial, Container-Images oder Build-Artefakte deutlich performanter gespeichert und gelesen werden.

Auch externe Benchmarks bestätigen den deutlichen Leistungssprung. In Geekbench-6-Messungen erreicht der Raspberry Pi 5 im Single-Core-Test etwa 764 bis 774 Punkte, während der Raspberry Pi 4 bei ungefähr 340 Punkten liegt. Dies entspricht etwa dem 2,4-fachen Leistungsniveau [1, Abschnitt “Single core”]. Zusätzlich werden dem Raspberry Pi 5 im Vergleich zum Raspberry Pi 4 eine höhere Speicher- und Secure Digital (SD)-Karten-Performance sowie eine verbesserte Kamera- und I/O-Anbindung zugeschrieben [22, Abschnitte “Performance” sowie “PCI, Cameras, Displays...”].

Tabelle 1: Direkter Hardwarevergleich zwischen Raspberry Pi 4 und Raspberry Pi 5

Merkmal	Raspberry Pi 4	Raspberry Pi 5
CPU	Cortex-A72	Cortex-A76
Arbeitsspeicher	LPDDR4, bis zu 8 GB	LPDDR4X
PCIe	nicht vorhanden	PCIe 2.0 x1
I/O-Architektur	klassisch integriert	RP1-I/O-Controller
USB-Bandbreite	geringer	höher
Kameraanbindung	geringer	verbessert
RTC	nicht hervorgehoben	auf dem Board vorhanden
Power-Button	nicht hervorgehoben	auf dem Board vorhanden

Für ein autonomes Fahrzeug ist der Raspberry Pi 5 vor allem aufgrund seiner höheren CPU-Reserven und verbesserten I/O-Fähigkeiten geeigneter. Sensorfusion, Ka-

merabildverarbeitung und mehrere parallel laufende Softwarekomponenten profitieren stark von dieser zusätzlichen Leistung. Der Raspberry Pi 4 bleibt hingegen eine sinnvolle Option für bestehende Projekte oder kostenkritische Anwendungen, wenn die Anforderungen an Bildverarbeitung, Datenrate und Erweiterbarkeit begrenzt bleiben.

Bei der Energieversorgung ist zwischen Netzteilanforderung und tatsächlicher Leistungsaufnahme zu unterscheiden. Für den Raspberry Pi 5 nennt der Hersteller eine Stromversorgung über USB-C mit 5 Volt (V) und 5 Ampere (A) über USB Power Delivery [18, Abschnitt "Specification"]. Für den Raspberry Pi 4 Model B werden 5 V und 3 A über USB-C oder GPIO angegeben; ein gutes 2,5-A-Netzteil kann genügen, wenn die USB-Peripherie weniger als 500 Milliampere (mA) benötigt [19, Abschnitt "Power requirements"]. Diese Angaben sind jedoch Versorgungsanforderungen und dürfen nicht unmittelbar als durchschnittlicher Energieverbrauch interpretiert werden.

3.2 Vergleich unterschiedlicher Betriebssysteme für Raspberry Pi

Die Wahl des Betriebssystems beeinflusst Ressourcenverbrauch, Hardwareintegration, ROS2-Kompatibilität, Wartbarkeit und Entwicklungsaufwand. Für ein autonomes Fahrzeug sind insbesondere schlanke Systeme mit guter Treiberunterstützung und stabiler Langzeitnutzung relevant. Betrachtet werden Raspberry Pi OS Lite, Ubuntu Server, Ubuntu Desktop und DietPi. Zusätzlich wird die Bedeutung von ARM64 für ROS2 berücksichtigt.

Raspberry Pi OS Lite ist das offiziell naheliegende Betriebssystem für viele Raspberry-Pi-Anwendungsfälle. Es verzichtet auf eine Desktop-Umgebung und eignet sich daher besonders für Headless-Server, Embedded-Anwendungen und batteriebetriebene Systeme. Der geringe Ressourcenbedarf reduziert RAM-Verbrauch, Hintergrundlast und tendenziell auch den Energiebedarf. Raspberry Pi OS Lite 64 Bit wird für Raspberry Pi 3, 4 und 5 empfohlen und bietet eine sehr gute Hardwareintegration [20, Abschnitte "Raspberry Pi OS", "Raspberry Pi OS Full" und "Raspberry Pi OS Lite"] [21].

Ubuntu Server ist vor allem dann attraktiv, wenn das Projekt stark am Ubuntu- und ROS-Ökosystem ausgerichtet ist. Viele ROS2-Installationsanleitungen, Pakete und Entwicklungsabläufe beziehen sich direkt auf Ubuntu. Dadurch kann Ubuntu Server den Entwicklungsaufwand reduzieren, wenn bereits Linux-Server-Know-how vorhanden ist oder wenn professionelle DevOps-nahe Workflows genutzt werden sollen [16, Abschnitt

“Ubuntu”]. Ubuntu nennt als Systemanforderungen für Server-Installationen unter anderem einen Mindestbedarf an Arbeitsspeicher und Massenspeicher, wobei für produktive Robotikprojekte ausreichend schneller Speicher eingeplant werden sollte [24].

Ubuntu Desktop eignet sich besonders für Entwicklung, Simulation und Debugging direkt auf dem Raspberry Pi. Werkzeuge wie RViz und Gazebo können lokal genutzt werden, was bei der Entwicklung und Analyse von ROS2-Systemen hilfreich ist. Gleichzeitig benötigt Ubuntu Desktop deutlich mehr RAM, Speicherplatz und Rechenleistung als ein Headless-System. Für den produktiven Fahrzeugeinsatz ist es daher weniger geeignet, kann aber als Entwicklungsplattform sinnvoll sein [3].

DietPi ist eine besonders schlanke Alternative für ressourcenschonende Embedded- oder Server-Setups. Es kann interessant sein, wenn minimale Systemlast wichtiger ist als maximale offizielle Raspberry-Pi-Hardwareintegration. Da in der übergebenen BibTeX-Datei jedoch kein eigener DietPi-Eintrag enthalten ist, sollte für eine endgültige wissenschaftliche Fassung noch ein passender BibTeX-Eintrag ergänzt werden.

Für ROS 2 ist Advanced RISC Machine 64-bit (ARM64) besonders relevant, da diese Architektur im Raspberry-Pi-Kontext eine wichtige Rolle spielt. Eine direkte native ROS 2-Installation ist insbesondere auf Ubuntu naheliegend. Raspberry Pi OS 64 Bit kann dagegen in Kombination mit Docker genutzt werden, um Ubuntu-basierte ROS 2-Container auszuführen [15].

Tabelle 2: Vergleich relevanter Betriebssystemoptionen

Kriterium	Pi OS Lite	Ubuntu Server	Ubuntu Desktop	DietPi
RAM-Bedarf	sehr gering	mittel	hoch	sehr gering
Strombedarf	gering	mittel	hoch	gering
ROS 2-Nähe	gut	sehr gut	sehr gut	Setupabhängig
Hardwareintegration	sehr gut	gut	gut	gut bis mittel
Entwicklung am Gerät	eingeschränkt	eingeschränkt	sehr gut	eingeschränkt
SSD/NVMe-Einsatz	optional sinnvoll	sinnvoll	sehr sinnvoll	optional

Für ein autonomes Fahrzeug mit Batteriebetrieb und begrenzten Ressourcen ist Raspberry Pi OS Lite 64 Bit besonders geeignet. Es bietet eine gute Kombination aus geringer Systemlast und sehr guter Hardwareunterstützung. Ubuntu Server bleibt eine starke Alternative, wenn ROS2-Kompatibilität, vorhandenes Ubuntu-Know-how oder spätere Skalierung im Vordergrund stehen. Ubuntu Desktop sollte primär als Entwicklungs- und Debugging-Umgebung betrachtet werden.

3.3 Softwarearchitektur

Die Softwarearchitektur legt fest, wie Sensorik, Aktorik, Wahrnehmung, Steuerung und Diagnose logisch strukturiert und miteinander verbunden werden. Für autonome Fahrzeuge bietet sich eine modulare Architektur an, bei der einzelne Funktionen in getrennten Softwarekomponenten ausgeführt werden. Dadurch können Sensorverarbeitung, Lokalisierung, Fahrentscheidung und Motorsteuerung unabhängig entwickelt, getestet und gewartet werden.

ROS 2 eignet sich als zentrale Middleware für ein solches System. Es unterstützt die modulare Kommunikation zwischen Knoten und ist in Robotik, autonomen Fahrzeugsystemen und Forschungsprojekten weit verbreitet. Sensoren, Aktoren, Steuerungslogik und Diagnosefunktionen können in ROS 2 als getrennte Nodes realisiert werden. Die Kommunikation erfolgt über Topics, Services und Actions, wodurch eine klare Trennung zwischen Datenströmen, Steuerbefehlen und länger laufenden Aktionen möglich wird [15].

Docker kann ergänzend eingesetzt werden, um Softwareabhängigkeiten und Laufzeitumgebungen in Container zu kapseln. Dies verbessert Reproduzierbarkeit, Wartbarkeit und Deployment, insbesondere wenn mehrere Entwickler beteiligt sind oder mehrere Softwarekomponenten unabhängig voneinander aktualisiert werden müssen. Für kleinere Einzelentwicklerprojekte kann Docker jedoch zusätzliche Komplexität verursachen.

Wissenschaftliche Untersuchungen zeigen, dass Linux-Container bei CPU- und Speicheroperationen häufig nahezu native Performance erreichen können, während relevanter Overhead vor allem bei I/O- und Netzwerkpfaden entstehen kann [8, Edition "An Updated Performance Comparison of Virtual Machines and Linux Containers"]. Spezifische Untersuchungen auf verschiedenen Raspberry-Pi-Modellen zeigen zudem, dass Docker grundsätzlich auch auf ressourcenbegrenzten Raspberry-Pi-Systemen einsetzbar ist. Die praktische Skalierbarkeit hängt jedoch stark vom verfügbaren Arbeitsspeicher und von der jeweiligen Container-Workload ab [6].

Der Ressourcenverbrauch von Containern sollte projektbezogen gemessen werden. Docker stellt hierfür unter anderem Metriken zu CPU-Auslastung, Speicherverbrauch, Netzwerk-I/O und Block-I/O bereit [5]. Außerdem verursacht Docker keinen festen, pauschalen Random Access Memory (RAM)-Overhead pro Container. Der tat-

sächliche Ressourcenbedarf hängt vom Container-Image, von der laufenden Anwendung, von der Anzahl paralleler Container und von gesetzten Ressourcenlimits ab [4].

Tabelle 3: Vergleich zwischen nativer ROS2-Installation und Docker-basierter ROS2-Architektur

Kriterium	ROS 2 nativ	Docker + ROS 2
Energieeffizienz	besser	tendenziell schlechter
RAM-Verbrauch	geringer	höher bzw. workloadequabhängig
Deployment	aufwendiger	einfacher reproduzierbar
Wartbarkeit	geringer	höher
Teamarbeit	weniger komfortabel	besser geeignet
Reproduzierbarkeit	geringer	höher
Hardwarezugriff	direkter	konfigurationsabhängig
Komplexität	geringer	höher

Für ein Studienprojekt oder ein Einzelentwicklerprojekt überwiegen häufig die Vorteile einer nativen ROS 2-Installation: geringere Komplexität, direkter Hardwarezugriff und weniger Ressourcenbedarf. Docker lohnt sich insbesondere dann, wenn reproduzierbare Deployments, klare Trennung mehrerer Komponenten, Teamarbeit oder Continuous Integration/Continuous Deployment (CI/CD)-Prozesse relevant sind.

3.4 Betrachtete Systemarchitekturen

Aus der Kombination von Hardware, Betriebssystem, ROS2 und optionaler Containerisierung ergeben sich mehrere technisch sinnvolle Systemarchitekturen. Diese unterscheiden sich insbesondere hinsichtlich Ressourcenbedarf, Wartbarkeit, Entwicklungsaufwand und Eignung für den produktiven Fahrzeugeinsatz.

Die erste Variante ist Raspberry Pi OS Lite 64 Bit mit nativer ROS 2-Installation. Diese Architektur ist besonders ressourcenschonend und hardwarenah. Sie eignet sich für Einzelentwicklungen, Studienprojekte und batteriebetriebene Fahrzeuge, bei denen geringe Systemlast und einfache Wartung wichtiger sind als vollständig containerisierte Deployments. Der direkte Hardwarezugriff kann insbesondere bei GPIO, Kamera, Sensorik und Aktorik vorteilhaft sein.

Die zweite Variante ist Raspberry Pi OS Lite 64 Bit mit Docker und ROS 2. Sie kombiniert die gute Raspberry-Pi-Hardwareintegration mit reproduzierbaren Softwareumgebungen. Diese Architektur ist sinnvoll, wenn mehrere ROS 2-Komponenten sauber voneinander getrennt werden sollen oder wenn ein konsistentes Deployment auf meh-

rerer Geräten erforderlich ist. Der Nachteil liegt im höheren Administrationsaufwand und im zusätzlichen Ressourcenbedarf.

Die dritte Variante ist Ubuntu Server mit nativer ROS 2-Installation. Sie ist besonders geeignet, wenn das Projekt stark am Ubuntu- und ROS 2-Ökosystem orientiert ist. Viele ROS 2-Anleitungen und Pakete sind für Ubuntu ausgelegt, wodurch die Einrichtung erleichtert werden kann [15]. Gegenüber Raspberry Pi OS Lite ist jedoch mit einer höheren Systemlast zu rechnen.

Die vierte Variante ist Ubuntu Server mit Docker und ROS 2. Diese Architektur bietet hohe Modularität und Reproduzierbarkeit. Sie ist besonders für Teamprojekte, CI/CD-Prozesse und verteilte Softwarekomponenten geeignet. Gleichzeitig ist sie die komplexeste der betrachteten produktiven Varianten und benötigt sorgfältige Ressourcenüberwachung, insbesondere auf einem Raspberry Pi [6] [5].

Die fünfte Variante ist Ubuntu Desktop mit ROS 2. Sie eignet sich vor allem als Entwicklungsplattform für grafische Werkzeuge wie RViz oder Gazebo. Für den produktiven Fahrzeugeinsatz ist sie wegen des höheren RAM-, Speicher- und Energiebedarfs weniger attraktiv [3].

Tabelle 4: Bewertung betrachteter Systemarchitekturen

Architektur	Stärken	Einschränkungen
Pi OS Lite + ROS 2	effizient, hardwarenah	weniger reproduzierbares Deployment
Pi OS Lite + Docker + ROS 2	wartbar, reproduzierbar	mehr Komplexität und Ressourcenbedarf
Ubuntu Server + ROS 2	gute ROS 2-Nähe	höhere Systemlast
Ubuntu Server + Docker + ROS 2	modular, teamfähig	komplexeste Variante
Ubuntu Desktop + ROS 2	gut für Entwicklung	ungeeignet für ressourcenarmen Betrieb

3.5 Kommunikationskonzept

Das Kommunikationskonzept beschreibt, wie auf das Fahrzeug zugegriffen wird und wie Steuerung, Diagnose und Wartung erfolgen. Für ein autonomes Fahrzeug ist eine robuste und möglichst unabhängige Kommunikationslösung wichtig, da das System auch außerhalb vorhandener Netzwerkinfrastruktur zuverlässig erreichbar bleiben soll.

Ein lokaler WLAN-Access-Point auf dem Raspberry Pi stellt hierfür eine geeignete

Lösung dar. Das Fahrzeug baut dabei ein eigenes WLAN auf, mit dem sich ein Entwicklungsrechner, Tablet oder Smartphone verbinden kann. Dadurch sind Steuerung, Parametrierung, Diagnose und Logzugriff auch dann möglich, wenn kein externes Netzwerk vorhanden ist. Diese Betriebsart ist besonders praktisch für Tests in Laborumgebungen, auf Teststrecken oder in Bereichen ohne zuverlässige WLAN-Infrastruktur.

Alternativ kann der Raspberry Pi als WLAN-Client in ein vorhandenes Netzwerk eingebunden werden. Dies ist sinnvoll, wenn das Fahrzeug dauerhaft in einer Labor- oder Campusinfrastruktur betrieben wird. Der Vorteil liegt in der einfachen Integration in bestehende Netzwerke. Der Nachteil besteht darin, dass die Erreichbarkeit von der vorhandenen Infrastruktur abhängt.

Ethernet bietet eine robuste und latenzarme Verbindung, ist jedoch kabelgebunden. Es eignet sich daher besonders für Laborbetrieb, stationäre Tests, Einrichtung und Debugging, aber weniger für den mobilen Fahrzeugeinsatz. LTE oder 5G ermöglichen entfernte Einsätze und Fernzugriff, erhöhen jedoch Kosten, Energiebedarf und Systemkomplexität. Für ROS2-Systeme ist außerdem zu beachten, dass die zugrunde liegende DDS-Kommunikation netzwerkabhängig konfiguriert werden muss, insbesondere wenn mehrere Systeme über bestehende Infrastruktur miteinander kommunizieren.

Für das betrachtete autonome Fahrzeug ist daher ein lokaler WLAN-Access-Point als primäres Kommunikationskonzept sinnvoll. Ergänzend kann Ethernet für Einrichtung und Debugging vorgesehen werden. WLAN-Client-Betrieb oder Mobilfunk können optional ergänzt werden, wenn der konkrete Anwendungsfall dies erfordert.

3.6 Entscheidung und Begründung

Auf Basis des Hardwarevergleichs, der Betriebssystembewertung und der betrachteten Softwarearchitekturen wird folgende Zielplattform empfohlen:

- Raspberry Pi 5 als Hardwareplattform,
- Raspberry Pi OS Lite 64 Bit als Betriebssystem,
- ROS 2 als zentrale Robotik-Middleware,
- Docker optional, abhängig von Teamgröße, Deployment-Anforderungen und Projektkomplexität,

- lokaler WLAN-Access-Point als primäres Kommunikationskonzept.

Der Raspberry Pi 5 wird gegenüber dem Raspberry Pi 4 bevorzugt, da er deutlich höhere CPU-Reserven, bessere I/O-Leistung, eine modernere Speicheranbindung und PCIe-Erweiterbarkeit bietet [18, Abschnitte “Overview” und “Specification”]. Gerade bei Sensorfusion, Kamerabildverarbeitung und parallelen ROS 2-Knoten sind diese Reserven entscheidend. Die gemessenen Benchmark-Unterschiede zeigen zudem, dass der Raspberry Pi 5 im Single-Core-Bereich deutlich leistungsfähiger ist als der Raspberry Pi 4 [1, Abschnitt “Single core”].

Raspberry Pi OS Lite 64 Bit wird gewählt, weil es eine geringe Systemlast mit sehr guter Hardwareintegration verbindet. Der Verzicht auf eine Desktop-Umgebung reduziert RAM-Verbrauch, Hintergrundprozesse und Energiebedarf. Dies ist für ein batteriebetriebenes autonomes Fahrzeug besonders wichtig [20, Abschnitte “Raspberry Pi OS” und “Raspberry Pi OS Lite”].

ROS 2 wird als Softwarebasis gewählt, weil es eine modulare Architektur für Robotiksysteme ermöglicht und die Kommunikation zwischen Sensorik, Aktorik, Wahrnehmung und Steuerung strukturiert unterstützt [15]. Die native Installation ist für ein Einzelentwickler- oder Studienprojekt zunächst die effizienteste Lösung, da sie weniger Systemkomplexität verursacht und direkten Hardwarezugriff erleichtert.

Docker wird nicht als zwingender Bestandteil der Zielarchitektur festgelegt, sondern als optionale Ergänzung bewertet. Der Einsatz ist sinnvoll, wenn mehrere Entwickler beteiligt sind, mehrere ROS 2-Komponenten sauber getrennt werden sollen oder reproduzierbare Deployments erforderlich sind. Wenn hingegen maximale Akkulaufzeit, maximale Performance oder ein möglichst einfaches System im Vordergrund stehen, ist eine native ROS 2-Installation vorzuziehen. Diese differenzierte Entscheidung ist wichtig, da Docker zwar Wartbarkeit und Reproduzierbarkeit verbessert, aber zusätzlichen Administrationsaufwand und workloadequabhängigen Ressourcenbedarf verursacht [6] [4].

3.7 Fazit

Der Technikkorridor zeigt, dass der Raspberry Pi 5 mit Raspberry Pi OS Lite 64 Bit und ROS 2 den besten technologischen Ausgangspunkt für ein autonomes Fahrzeug im Studien- oder Forschungsumfeld darstellt. Diese Kombination verbindet hohe Rechen-

und I/O-Reserven mit geringer Systemlast, guter Hardwareunterstützung und hoher Zukunftssicherheit.

In der praktischen Umsetzung des Projekts wurden jedoch Anpassungen an dieser Empfehlung vorgenommen. Statt Raspberry Pi OS Lite wird Ubuntu Server 22.04 LTS als Betriebssystem eingesetzt. Diese Entscheidung wurde getroffen, um eine maximale Kompatibilität mit dem ROS 2-Ökosystem sicherzustellen, da viele ROS 2-Pakete und Entwicklungswerkzeuge primär für Ubuntu entwickelt und getestet werden. Dies vereinfacht den Entwicklungs- und Integrationsprozess erheblich.

Des Weiteren kommt für die Implementierung und die in dieser Arbeit dargestellten Ergebnisse der Raspberry Pi 4 zum Einsatz. Obwohl der Raspberry Pi 5 die leistungsfähigere Plattform darstellt, traten während der Projektlaufzeit unvorhergesehene technische Herausforderungen bei der Integration auf. Um die Stabilität des Gesamtsystems und den Projektfortschritt nicht zu gefährden, wurde daher auf die bewährte und stabile Plattform des Raspberry Pi 4 zurückgegriffen. Die im Technikkorridor erarbeiteten Erkenntnisse bezüglich des Raspberry Pi 5 behalten jedoch ihre Gültigkeit als Empfehlung für zukünftige Ausbaustufen oder ähnliche Projekte.

Der Raspberry Pi 4 bleibt für bestehende oder kostenkritische Projekte weiterhin nutzbar, ist für leistungsorientierte autonome Systeme jedoch weniger geeignet. Der Raspberry Pi 5 bietet durch seine stärkere CPU, den RP1-I/O-Controller, bessere Kamera- und USB-Anbindung sowie PCIe/NVMe-Erweiterbarkeit die deutlich tragfähigere Plattform [18, Abschnitte "Overview" und "Specification"] [22, Abschnitte "Performance" sowie "PCI, Cameras, Displays..."].

Die Softwarearchitektur basiert wie empfohlen auf ROS 2. Auch das Kommunikationskonzept wurde wie im Technikkorridor empfohlen umgesetzt. Es wird ein lokaler WLAN-Access-Point als primäre Kommunikationslösung verwendet, da diese für den mobilen Einsatz im Rahmen des Projekts am flexibelsten ist. Die ebenfalls diskutierten Optionen Ethernet und LTE/5G wurden für den in dieser Arbeit betrachteten Anwendungsfall als überdimensioniert bewertet und daher nicht weiterverfolgt.

Docker verbessert Wartbarkeit, Reproduzierbarkeit und Skalierbarkeit, sollte aber nicht pauschal als Pflichtbestandteil der Architektur betrachtet werden. Für Einzelentwicklungen und batteriebetriebene Studienprojekte ist Raspberry Pi OS Lite mit nativer ROS 2-Installation meist die effizienteste Lösung. Docker lohnt sich insbesondere bei Teamprojekten, CI/CD-Prozessen oder mehreren verteilten Softwarekomponenten.

Der empfohlene Technikkorridor lautet daher: Raspberry Pi 5, Raspberry Pi OS Lite 64 Bit, ROS 2 nativ als Basis und Docker als optionale Erweiterung.

4 Methodik

Im Rahmen dieser wissenschaftlichen Arbeit werden verschiedene Methoden angewendet, um eine fundierte und nachvollziehbare Lösung zu erarbeiten. Ein zentrales Instrument hierbei ist die Nutzwertanalyse, die zur systematischen Entscheidungsfindung bei der Auswahl von Hard- und Softwarekomponenten dient. Diese Methode ermöglicht es, verschiedene Optionen anhand definierter Kriterien objektiv zu bewerten und die für den Anwendungsfall am besten geeignete Lösung zu identifizieren.

4.1 Nutzwertanalyse

			Option 1		Option 2		Option 3	
	Wertebereich	Gewichtung	Bewertung	Nutzwert	Bewertung	Nutzwert	Bewertung	Nutzwert
Kriterium 1	0-5	10	1	10	3	30	3	30
Kriterium 2	0-5	30	2	60	5	150	2	60
Kriterium 3	0-5	25	4	100	1	25	4	100
Kriterium 4	0-5	25	3	75	2	50	3	75
Kriterium 5	0-5	15	3	45	2	30	2	30
Nutzwertsumme				290		285		295
Rangplatz				2		3		1

Tabelle 5: Beispiel Nutzwertanalyse

Eine Nutzwertanalyse (siehe Tabelle 5 vgl. [25]) ist eine Möglichkeit, aus vielen Optionen, die bestimmte Kriterien erfüllen müssen, die beste Option zu finden. Sie ist in verschiedene Spalten aufgeteilt. Die linke Spalte enthält die Kriterien, welche erfüllt sein müssen, damit das Endergebnis gut ist. Die folgende Spalte enthält den Wertebereich, der verteilt werden kann. Spalte drei kann optional eine Gewichtung der Kriterien festlegen, dadurch können genauere Ergebnisse erzielt werden. Diese Nutzwertanalyse wäre in dem Fall eine gewichtete Nutzwertanalyse, ohne Gewichtung wäre es nur eine einfache Nutzwertanalyse. Anschließend an die ersten Spalten kommen nun die Optionsspalten. Diese werden Spalte für Spalte durchgearbeitet und Punkte vergeben.

Falls hinter Punkten eine gewisse Bedeutung liegt, ist die Erstellung einer Legende notwendig. Für das Ergebnis werden nun die Eintragungen mit der Gewichtung der Zeile multipliziert und dann die Optionsspalte addiert. Anhand des Ergebnisses kann dann ausgewertet werden, welche Option am besten die Kriterien erfüllt. vgl. [25, S. 9–11]

Die farbliche Abhebung wird genutzt, damit schneller erkenntlich ist, welche Option ein Kriterium am besten erfüllt.

5 Experimentelles Evaluationsdesign

Zur Bewertung der entwickelten Lösung wurden standardisierte Testszenarien definiert. Ziel war die Untersuchung der Zuverlässigkeit der Hinderniserkennung sowie der autonomen Spurplanung. Hierfür wurden drei Testszenarien festgelegt:

- Szenario A: Einzelnes Hindernis mittig auf der Fahrbahn
- Szenario B: Hindernis rechts versetzt zur Fahrbahn
- Szenario C: Zwei aufeinanderfolgende Hindernisse

Jedes Szenario wurde 10-mal durchgeführt. Erfasst wurden die Erfolgsquote der Hinderniserkennung sowie die Erfolgsquote des Ausweichmanövers. Die Ergebnisse werden in Kapitel 10 quantitativ ausgewertet und mit den zuvor definierten Projektzielen verglichen.

5.1 Methoden zur Hindernisumfahrung

5.1.1 Mathematische Formeln für die Berechnung der Fahrbahn

Bei der mathematischen Modellierung von Trajektorien zur Umfahrung eines Hindernisses spielt die Wahl der zugrunde liegenden Funktion eine entscheidende Rolle. Insbesondere sind Aspekte wie Krümmungsverhalten, lokale Anpassungsfähigkeit und das asymptotische Verhalten von großer Bedeutung. Im Folgenden werden Parabeln, Exponentialfunktionen und die Gaußsche Glockenkurve im Hinblick auf ihre Eignung zur Umfahrung eines Gegenstandes verglichen.

Parabeln

Eine Parabel ist eine ganzrationale Funktion zweiten Grades der Form

$$f(x) = ax^2 + bx + c.$$

Sie besitzt genau einen Scheitelpunkt und eine konstante zweite Ableitung. Diese konstante Krümmung führt dazu, dass das Auto nicht gerade auf einen Gegenstand zu fahren kann, ihn umfahren und wieder zurück auf seine Ausgangsspur zurück fahren

kann nach dem Umfahren. Eine Parabel wäre daher nur bedingt geeignet um vor einem Gegenstand umzudrehen falls der Gegenstand nicht umfahrbar ist.

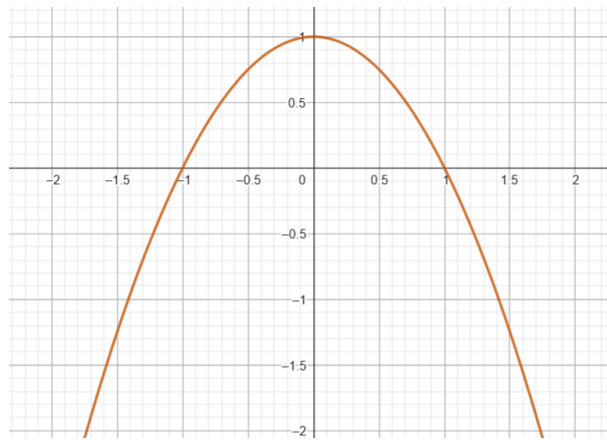


Abbildung 1: Parabel Funktion

Exponentialfunktionen

Exponentialfunktionen der Form

$$f(x) = a \cdot e^{bx}$$

zeigen ein stark asymmetrisches Verhalten. Sie besitzen keinen Scheitelpunkt und keine natürliche Symmetrieachse. Das Wachstum bzw. der Abfall erfolgt einseitig sehr stark, während auf der anderen Seite nur eine geringe Änderung stattfindet. Für die Umfahrung eines Gegenstandes ist diese Eigenschaft nachteilig, da eine ausgewogene seitliche Auslenkung benötigt wird. Exponentialfunktionen erzeugen entweder eine zu abrupte oder eine zu flache Richtungsänderung und erlauben keine kontrollierte Rückführung auf die ursprüngliche Spur. Allerdings könnten sie in speziellen Fällen, z.B. bei sehr engen Hindernissen, als Notfallmanöver eingesetzt werden, um schnell Kurven zu fahren, wenn keine andere Option besteht.

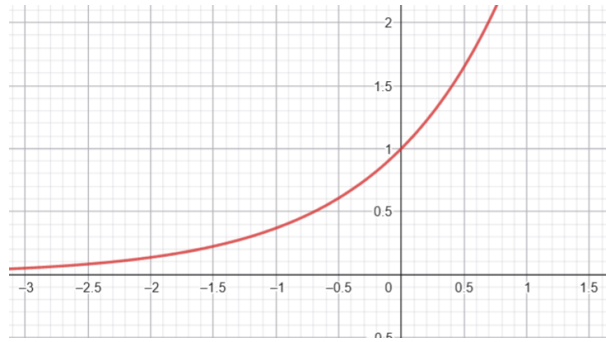


Abbildung 2: Exponential Funktion

Gaußsche Glockenkurve

Die Gaußsche Glockenkurve ist definiert durch

$$f(x) = A \cdot e^{-\frac{(x-\mu)^2}{2\sigma^2}}.$$

Sie besitzt einen klar definierten Scheitelpunkt bei $x = \mu$, ist streng symmetrisch und fällt beidseitig sanft gegen null ab. Diese Eigenschaften machen sie besonders geeignet für die Umfahrung von Hindernissen. Die Ausweichbewegung ist lokal begrenzt, kontinuierlich differenzierbar und verursacht keine abrupten Richtungsänderungen. Gleichzeitig wird der ursprüngliche Fahrweg nach dem Umfahren des Hindernisses automatisch wieder angenähert. Ein weiterer entscheidender Vorteil der Gaußschen Glocken-

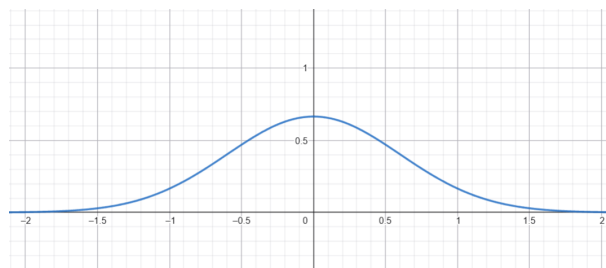


Abbildung 3: Gaussche Glocke Funktion

kurve ergibt sich durch ihre rekursive Anwendung. Wird nach der Umfahrung eines Hindernisses am Scheitelpunkt der resultierenden Kurve erneut eine Gaußfunktion angesetzt, so können auch komplexe Hindernisformen oder mehrere aufeinanderfolgende Objekte zuverlässig umfahren werden. Durch diese rekursive Überlagerung entsteht eine adaptive Fahrtenkurve, die sich flexibel an nahezu jede Umgebung anpassen kann. Unter geeigneten Randbedingungen kann die rekursive Anwendung mehrerer Gaußfunktionen die Umfahrung komplexer Hindernisstrukturen unterstützen.

Fazit

Zusammenfassend lässt sich sagen, dass die Gaußsche Glockenkurve aufgrund ihrer symmetrischen Form, der lokalen Begrenzung, einfachen Parametrierbarkeit und der sanften Krümmung besonders gut geeignet ist für die Umfahrung von Gegenständen. Parabeln können in bestimmten Situationen nützlich sein, bieten jedoch nicht die notwendige Flexibilität und Kontrolle. Exponentialfunktionen sind für diesen Anwendungsfall weniger geeignet, da sie zu einseitig und abrupt reagieren. Die Möglichkeit der rekursiven Anwendung der Gaußschen Glockenkurve ermöglicht es zudem, komplexe Hindernisformationen effektiv zu bewältigen, was sie zur optimalen Lösung für die Fahrbahnplanung bei der Umfahrung von Gegenständen macht. Damit ein optimales Fahrverhalten erreicht wird, sollten die Parameter der Gaußschen Glockenkurve sorgfältig angepasst werden, um eine harmonische und sichere Umfahrung zu gewährleisten. Dabei ist wichtig zu beachten, dass die Wahl der Kurvensteigung nicht über den maximalen Lenkeinschlag hinausgehen darf, um die Fahrstabilität zu gewährleisten. Insgesamt bietet die Gaußsche Glockenkurve eine robuste und flexible Grundlage für die Entwicklung von Algorithmen zur Hindernisumfahrung in autonomen Fahrsystemen.

5.1.2 Berechnung Lenkwinkel

Die Berechnung des Lenkwinkels für die Umfahrung eines Gegenstandes basiert auf der geometrischen Beziehung zwischen der Fahrbahnkurve und der Fahrzeugdynamik. Angenommen, die Fahrbahn wird durch eine Funktion $f(x)$ beschrieben, die die seitliche Auslenkung in Abhängigkeit von der Vorwärtsbewegung x angibt, kann der Lenkwinkel δ wie folgt approximiert werden:

$$\delta(x) = \arctan \left(\frac{df}{dx} \right).$$

Hierbei ist $\frac{df}{dx}$ die erste Ableitung der Fahrbahnfunktion, die die Steigung der Kurve an einem bestimmten Punkt x angibt. Der Lenkwinkel δ wird in Radiant oder Grad angegeben und gibt an, wie stark das Fahrzeug in diesem Moment lenken muss, um der Fahrbahn zu folgen. Die Berechnung des Lenkwinkels ist entscheidend für die Steuerung des Fahrzeugs und die Gewährleistung einer sicheren Umfahrung des Gegenstandes. Es ist wichtig, dass der berechnete Lenkwinkel innerhalb der mechanischen Grenzen

des Fahrzeugs liegt, um eine Übersteuerung oder Untersteuerung zu vermeiden. Zudem sollte die Berechnung in Echtzeit erfolgen, um auf dynamische Veränderungen in der Umgebung reagieren zu können. Die Implementierung dieser Berechnung erfordert eine genaue Modellierung der Fahrzeugdynamik und eine präzise Erfassung der Fahrbahnkurve, um eine effektive und sichere Umfahrung zu gewährleisten.

6 Autoumbau

6.1 Umverteilung des Gewichtes

Da der Lidar Sensor eine Neigung um 7° nach vorne hat (schwerer, ungestützter Vorbau), muss das Gewicht neu verteilt werden. Hierfür werden drei Möglichkeiten herausgearbeitet und in Tabelle 6 verglichen.

6.1.1 Hochkante Einlage in den Mittelteil

Art der Änderung

- Der Vorderbau wird mechanisch begradigt: die Bereiche um die Schraubpunkte werden abgefeilt, bis eine rechteckige, sauber einpassbare Form entsteht.
- Der Vorderbau wird anschließend hochkant in den Mittelteil integriert.
- Fixierung über einen Winkel hinter der Servolenkung, damit die Einheit verwindungssteif sitzt und Kräfte definiert eingeleitet werden.
- Das Batteriefach wird hochkant ausgerichtet und in Richtung Motor verschoben, um den Bauraum besser zu nutzen und den Schwerpunkt sinnvoll zu verlagern.
- Leitungsführung und Befestigungspunkte müssten dabei neu geplant werden, weil sich Einbauwinkel und Zugänglichkeit ändern.

Vorteile

- Sehr platzsparende Integration, kaum toter Bauraum.
- Wendekreis bleibt im Normalfall unverändert, da Radstand und Lenkeinschlag nicht zwangsläufig angepasst werden müssen.
- Konstruktion wirkt insgesamt kompakter und aufgeräumter, was spätere Erweiterungen erleichtern kann.

Nachteile

- Wartbarkeit kann schlechter werden: Akkuwechsel, Schrauben nachziehen, Komponenten tauschen wird je nach Einbauposition fummeliger.
- Höherer mechanischer Anpassungsaufwand (Feilen, Ausrichten, Winkel fertigen), dadurch mehr Risiko für Passungenauigkeiten.
- Kabelmanagement wird kritischer, weil weniger Platz für saubere Radien und Zugentlastung bleibt.

6.1.2 Frankenstein Umbau

Art der Änderung

- Vorderbau wird vollständig abgeschraubt.
- Mittelteil des Chassis wird durchgesägt, um eine neue Montageposition zu schaffen.
- Der Vorderbau wird in die Mitte versetzt und dort wieder verschraubt.
- Je nach Schnittstelle müssten Verstärkungen ergänzt werden (Platten, Winkel, zusätzliche Verschraubungen), damit die Struktur unter Last nicht nachgibt.

Vorteile

- Konzeptuell einfach, weil umsetzen und festschrauben als Grundidee klar ist.
- Kann schnell einen deutlichen Effekt auf Schwerpunkt und Platzverteilung bringen, ohne viele Detailarbeiten an einzelnen Bauteilen.

Nachteile

- Wendekreis kann schlechter werden, weil sich Geometrie und Gewichtsverteilung ungünstig verändern können.
- Stabilitätsrisiko: Durchsägen schwächt das Chassis, dadurch Verwindung, Risse, lockere Verschraubungen möglich.
- Oft mehr Nacharbeit als gedacht, weil nach dem groben Umbau Verstärkungen und Ausrichtung notwendig werden.

6.1.3 Einkaufsrad Lösung

Art der Änderung

- Ein Stützrad (Einkaufsrad) wird unter dem Vorderbau montiert.
- Ziel ist, dass das Rad einen Teil der Frontlast trägt und das Fahrzeug vorne mechanisch stabilisiert.
- Befestigung über eine einfache Halterung oder Montageplatte, ohne größere Eingriffe in den Aufbau.

Vorteile

- Sehr schnell und kostengünstig umsetzbar.
- Minimal invasiv: bestehender Aufbau bleibt weitgehend erhalten, Rückbau ist einfach.
- Sofortiger Effekt auf Lastverteilung, ohne Chassis oder Hauptbaugruppen umzubauen.

Nachteile

- Optisch eher Bastellösung, wirkt weniger integriert.
- Fahrverhalten kann schlechter werden: Nachlaufen, Flattern, zusätzliche Reibung, eingeschränkte Bodenfreiheit.
- Auf unebenem Untergrund oder bei Kanten kann das Stützrad stören oder hängen bleiben.

6.1.4 Nutzwertanalyse zur Gewichtsumverteilung

Aus der Nutzwertanalyse in Tabelle 6 ergibt sich, dass die Hochkante Einlage im Mittelteil die beste Lösung zur Umverteilung des Gewichtes darstellt. Die Lösung bietet eine gute Balance aus Umsetzbarkeit, Fahrverhalten und Wartbarkeit, während sie gleichzeitig den Schwerpunkt effektiv verlagert. Obwohl der mechanische Anpassungsaufwand höher ist, überwiegen die Vorteile in Bezug auf Platznutzung und Fahrzeugperformance. Daher wird diese Lösung für die weitere Umsetzung empfohlen.

			1. Hochkante Einlage in den Mittelteil		2. „Frankenstein“ Umbau		3. Einkaufsrat Lösung	
	Wertebereich	Gewichtung	Bewertung	Nutzwert	Bewertung	Nutzwert	Bewertung	Nutzwert
Umbau aufwand	0-5	5	3	15	1	5	5	25
Zeitlicher Aufwand	0-5	7	4	28	5	35	2	14
Optik	0-5	3	4	12	5	15	2	6
Vorteile	0-5	5	4	20	4	20	3	15
Nachteile	-5-0	5	-2	-10	-4	-20	-2	-10
Nutzwertsumme				65		55		50
Rangplatz				1		2		3

Tabelle 6: Nutzwertanalyse zur Gewichtsverteilung

7 Containerisierung und Systemstart

Das gesamte ROS 2 Backend wird in Docker-Containern [13] auf dem Raspberry Pi betrieben. Die Containerisierung verfolgt drei Ziele: eine reproduzierbare Laufzeitumgebung unabhängig vom Zustand des Host-Systems, identische Umgebungen auf Entwicklungsrechnern und Zielsystem sowie einen deterministischen Systemstart. Dieses Kapitel beschreibt die Ausgangslage mit ihren Problemen sowie die umgesetzten Lösungen.

7.1 Ausgangslage und Probleme

Das ursprüngliche, monolithische Dockerfile installierte Entwicklungswerkzeuge, grafische Anwendungen und Laufzeitabhängigkeiten in ein einziges Image. Daraus resultierten lange Bauzeiten und ein unnötig großes Image auf dem ressourcenbeschränkten Zielsystem. Durch den zu großen Container funktionierte auch die Anwendung nicht richtig: Das Übertragen und Entpacken des Images auf dem Raspberry Pi dauerte unverhältnismäßig lange, und die knappen Ressourcen des Zielsystems wurden durch nicht benötigte Bestandteile belastet.

Hinzu kam, dass der Start des Systems in frühen Projektphasen eine Abfolge manueller Schritte im Container erforderte. Der Startprozess musste erst verstanden und eingerichtet werden und war nicht intuitiv: Ohne Vorwissen über die richtige Reihenfolge

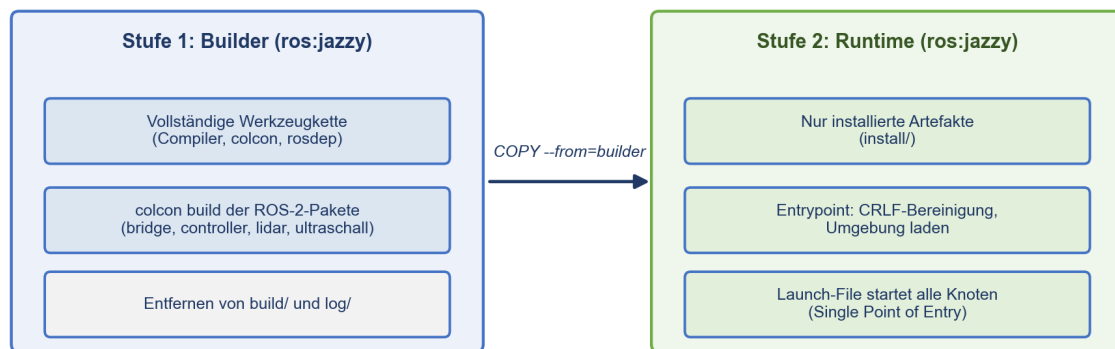
ge der Kommandos war das System nicht in Betrieb zu nehmen. Dies war fehleranfällig, erschwerte Testläufe und führte zu schwer reproduzierbaren Zuständen. Zusätzlich belasteten versehentlich eingeecheckte, generierte Bauartefakte das Repository und verursachten Konflikte zwischen den Arbeitsständen der Teammitglieder.

Ein weiteres Problem betraf die Zusammenarbeit zwischen Windows-Entwicklungsumgebung und Linux-Zielsystem: Skripte, die auf Windows-Rechnern ausgecheckt und anschließend in den Linux-Container eingebunden wurden, schlugen dort mit schwer interpretierbaren Fehlermeldungen fehl. Als Ursache wurden Windows-Zeileneenden (CRLF) identifiziert, die Git beim Auschecken auf Windows-Systemen standardmäßig einfügt. Der Linux-Interpreter wertet das zusätzliche Wagenrücklaufzeichen als Bestandteil von Befehlen beziehungsweise Pfaden, wodurch unter anderem die Ausführung des Entry-points und der Python-Knoten scheiterte. Das Problem ist besonders tückisch, weil es nur in der Kombination aus Windows-Checkout und Linux-Ausführung auftritt und auf dem Entwicklungsrechner nicht sichtbar ist.

Schließlich stand die Zielhardware, also das Fahrzeug mit Raspberry Pi, Sensorik und Aktorik, dem Team nur eingeschränkt zur Verfügung. Ohne Gegenmaßnahmen hätten alle hardwarenahen Knoten auf Entwicklungsrechnern nicht einmal gestartet werden können, da bereits der Import der GPIO-Bibliothek beziehungsweise der Zugriff auf Gerätedateien fehlschlägt.

7.2 Multi-Stage-Build

Zur Lösung des Größenproblems wurde der Bauprozess auf einen zweistufigen Multi-Stage-Build umgestellt, den Abbildung 4 veranschaulicht. Die erste Stufe, der Builder, enthält die vollständige Werkzeugkette und übersetzt die ROS-Pakete mit `colcon`. Zwischenartefakte wie Build- und Log-Verzeichnisse werden anschließend entfernt. Die zweite Stufe, die Runtime, basiert auf dem schlanken ROS-Basisimage und übernimmt ausschließlich die installierten Artefakte der Builder-Stufe. Ergänzend wurde eine `.dockerignore` eingeführt, damit nicht benötigte Dateien den Build-Kontext nicht aufblähen. Diese Struktur reduziert sowohl die Imagegröße als auch die Bauzeit deutlich und trennt Bau- und Laufzeitumgebung konzeptionell sauber. Die zuvor beobachteten Beeinträchtigungen der Anwendung durch das übergroße Image entfallen damit.



Ergebnis: deutlich kleineres Laufzeit-Image, kürzere Bauzeit, saubere Trennung von Bau- und Laufzeitumgebung

Abbildung 4: Zweistufiger Multi-Stage-Build der Container-Images

7.3 Deterministischer Systemstart

Der zuvor nicht intuitive Startprozess wurde vollständig automatisiert. Der Start des Containers wurde als einziger Einstiegspunkt (Single Point of Entry) gestaltet: Mit dem Start des Containers werden über ein zentrales Launch-File alle Knoten des Systems, also Bridge, Fahrzeugcontroller, LiDAR-Treiber, Sicherheitsknoten und Ultraschallsensor, automatisch und in definierter Konfiguration gestartet. Manuelle Startschritte auf dem Fahrzeug entfallen vollständig. Das System ist damit auch ohne Detailwissen über die interne Kommandofolge in Betrieb zu nehmen.

Für den Hardwarezugriff werden die benötigten Gerätedateien, also der GPIO-Chip und die serielle USB-Schnittstelle, explizit in den Container durchgereicht. Die WebSocket-Schnittstelle der Bridge wird über eine definierte Portzuordnung nach außen angeboten. Ein Portkonflikt der WebSocket-Schnittstelle wurde durch Umstellung der Portzuordnung behoben und der Arbeitsschritt im Repository dokumentiert. Die generierten Bauartefakte von in Summe circa 12700 Zeilen wurden aus der Versionsverwaltung entfernt und über erweiterte Ignorier-Regeln dauerhaft ausgeschlossen. Beide Maßnahmen verbesserten die Reproduzierbarkeit der Testläufe und die Stabilität der Zusammenarbeit erheblich.

7.4 Plattformübergreifende Lauffähigkeit

Die Lösung der Zeilenenden-Problematik wurde zweistufig ausgelegt. Präventiv erzwingt eine Regel in der Datei `.gitattributes (*.py eol=lf)`, dass Python-Dateien repositoryweit stets mit Unix-Zeilenenden ausgecheckt werden. Defensiv entfernt der Entrypoint des Containers beim Start zusätzlich vorhandene Wagenrücklaufzeichen aus den eingebundenen Skripten. Durch die Kombination beider Maßnahmen ist der Fehlerfall sowohl an der Quelle unterbunden als auch zur Laufzeit abgefangen.

Für die Entwicklung ohne verfügbare Zielhardware wurden alle hardwarenahen Knoten mit einem Simulationsrückfall ausgestattet. Beim Start wird die Verfügbarkeit der GPIO-Hardware aktiv geprüft, indem der Gerätezugriff probeweise geöffnet wird. Schlägt die Prüfung fehl, läuft der Knoten in einem Simulationsmodus weiter, in dem Hardwarezugriffe übersprungen und durch eine entsprechende Protokollmeldung kenntlich gemacht werden. Das Gesamtsystem bleibt dadurch auf beliebigen Entwicklungsrechnern lauffähig und testbar, was dem Prinzip der kontrollierten Degradation entspricht. Dies war eine wesentliche Voraussetzung für die arbeitsteilige Entwicklung im Team.

8 Implementierung Autonomes Fahren

8.1 Datenverarbeitung

Die Ist-Analyse der Sensorik hat gezeigt, dass zu Projektbeginn keine operative Anbindung realer LiDAR-Hardware existierte. Sämtliche LiDAR-bezogenen Komponenten arbeiteten ausschließlich auf statischen, vorab aufgezeichneten Datensätzen. Auf Basis dieses Befundes wurde eine vollständige Verarbeitungskette entworfen und implementiert, die den physischen Sensor in das ROS 2 System [10] integriert. Die Verarbeitungskette gliedert sich in drei Stufen: die Anbindung des Sensors über einen Treiberknoten, die Aufbereitung der Rohdaten in ein standardisiertes Nachrichtenformat sowie die sicherheitsgerichtete Auswertung der Daten im Fahrbetrieb.

8.1.1 Wechsel von der Ultraschallsensorik auf den LiDAR

Ursprünglich war der Ultraschallsensor als zentraler Abstandssensor des Fahrzeugs vorgesehen. Im Projektverlauf zeigte sich jedoch, dass die vorhandene Sensorik weitgehend nicht funktionierte, und die Ursachenanalyse ergab zwei voneinander unabhängige Probleme.

Das erste Problem betraf die GPIO-Ansteuerung insgesamt: Mit der Aktualisierung des OS auf dem Raspberry Pi brach die gesamte GPIO-Ansteuerung des Fahrzeugs, sodass Motorsteuerung, Lenkung, Status-LED und Ultraschallsensor nicht mehr funktionsfähig waren. Ursache war der Wechsel der Bibliothek `libgpiod` auf die Hauptversion 2, die einen bewusst inkompatiblen Schnittstellenbruch vollzieht. Die bis dahin genutzten Konstrukte `LineRequest()`, `get_line()` und `LINE_REQ_DIR_OUT` existieren in der neuen Version nicht mehr. Die Analyse der neuen Bibliotheksversion ergab ein grundlegend verändertes Programmiermodell: Statt einzelne Leitungsobjekte zu beziehen und separat zu konfigurieren, werden alle benötigten Leitungen in einem Aufruf mit ihren Einstellungen reserviert. Zusätzlich zeigte sich, dass der GPIO-Chip nicht mehr über den Kurznamen, sondern über den vollständigen Gerätepfad `/dev/gpiochip4` angesprochen werden muss. Zur Lösung wurden alle betroffenen Knoten, also Fahrzeugcontroller, Bridge und Ultraschallsensor, auf das neue Modell migriert: Die Leitungen werden zentral über `request_lines()` mit `LineSettings` für Richtung und Anfangszustand reserviert, Pegel werden über die expliziten Zustände `Value.ACTIVE` und

`Value.INACTIVE` gesetzt. Die im Projekt selbst implementierte Software-PWM zur Ansteuerung der Motoren wurde von der Arbeit auf einzelnen Leitungsobjekten auf das Paar aus Leitungsanforderung und Pin-Offset umgestellt. Die Migration wurde so gestaltet, dass das Verhalten der Fahrzeugsteuerung nach außen unverändert blieb.

Das zweite Problem betraf den Ultraschallsensor selbst: Auch nach der Migration lieferte er keine gültigen Messwerte, und die Reservierung des Trigger-Pins schlug mit einer Belegt-Meldung fehl. Die systematische Prüfung der Pinbelegung des Raspberry Pi ergab, dass der zunächst gewählte GPIO 7 hardwareseitig dem SPI-Subsystem als Chip-Select-Leitung zugeordnet ist und vom Betriebssystem belegt wird. Ein Zugriff über die GPIO-Schnittstelle ist damit blockiert. Das Trigger-Signal wurde daher auf den frei verfügbaren GPIO 26 (physischer Pin 37) verlegt und die Verdrahtung entsprechend angepasst. Ergänzend wurden die Zeitsteuerung der Messung überarbeitet und eine Diagnoseausgabe für ungültige Messungen ergänzt, um künftige Fehlerbilder schneller eingrenzen zu können.

Trotz dieser Reparaturen blieb die Ultraschallsensorik konzeptionell begrenzt: Sie liefert nur einen einzelnen Distanzwert in Blickrichtung und erlaubt damit weder eine Richtungszuordnung von Hindernissen noch eine Umgebungserfassung. Vor diesem Hintergrund wurde der Wechsel auf den LiDAR als primären Sensor für die Hinderniserkennung vollzogen: Der LiDAR erfasst die Umgebung in einer vollen 360-Grad-Drehung mit hunderten Messpunkten je Umlauf und gilt in der Literatur als Schlüsselsensorik für das autonome Fahren [9]. Mit der Inbetriebnahme des LiDAR-Treibers wurde der Ultraschallknoten deaktiviert. Er bleibt als redundante Rückfallebene für künftige Erweiterungen im Projekt erhalten.

8.1.2 Architektur der Verarbeitungskette

Die Architektur folgt dem Prinzip der strikten Trennung von Treiber- und Anwendungslogik. Der Treiberknoten `lidar_driver` ist ausschließlich für die Kommunikation mit der Hardware und die Publikation der Messdaten zuständig. Er trifft keinerlei Annahmen über die nachgelagerten Konsumenten. Die Verteilung der Daten erfolgt über das Publisher-Subscriber-Muster von ROS 2 [12] auf dem Topic `/scan` im standardisierten Nachrichtentyp `sensor_msgs/LaserScan`. Nachgelagerte Knoten, etwa die Hindernisvermeidung (siehe Abschnitt 8.2) oder die Not-Halt-Funktion, abonnieren dieses Topic

unabhängig voneinander. Diese Entkopplung erwies sich im Projektverlauf als tragfähig: Die gaußbasierte Ausweichlogik konnte ohne Änderungen am Treiber auf derselben Dateninfrastruktur aufgesetzt werden. Abbildung 5 zeigt die Komponenten und Datenflüsse im Überblick.

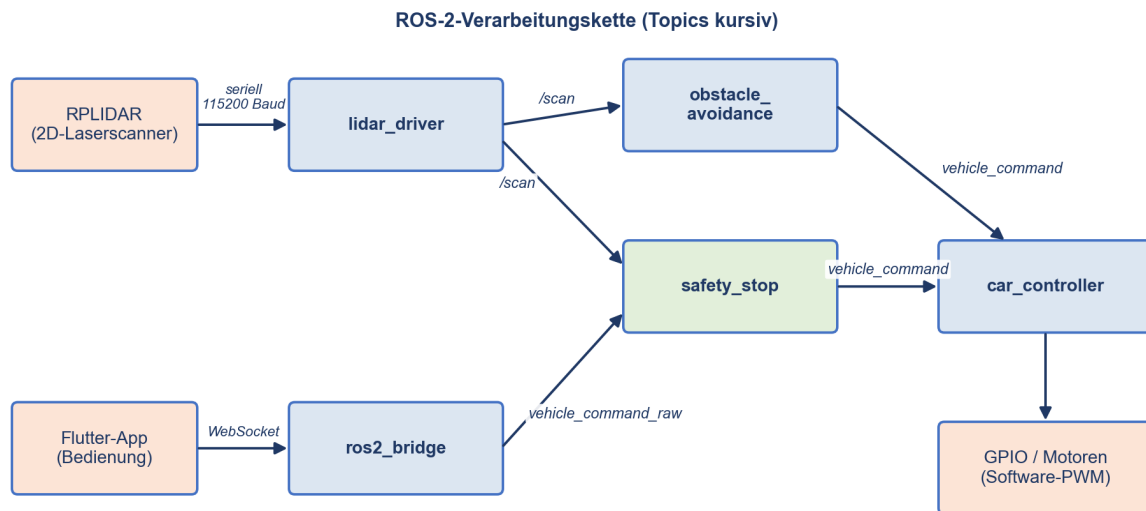


Abbildung 5: Komponenten und Datenflüsse der ROS 2 Verarbeitungskette

8.1.3 Treiberknoten und Aufbereitung der Scandaten

Der Treiberknoten liest den rotierenden 2D-Laserscanner (RPLIDAR) über die serielle Schnittstelle `/dev/ttyUSB0` mit einer Baudrate von 115200 aus. Das Auslesen erfolgt in einem separaten Hintergrund-Thread, damit der ROS-Executor nicht blockiert wird und der Knoten auf Steuerereignisse reagieren kann. Eine Wiederverbindungsschleife stellt den Betrieb nach Verbindungsabbrüchen automatisch wieder her, ohne dass ein manueller Eingriff erforderlich ist.

Die Rohmessungen des Sensors liegen als Tripel aus Signalqualität, Winkel und Distanz vor und treffen in unregelmäßiger Winkelfolge ein. Für die Weiterverarbeitung werden sie in ein regelmäßiges Raster aus 360 Winkelklassen mit einem Grad pro Klasse überführt. Fallen mehrere Messungen in dieselbe Klasse, wird die minimale Distanz übernommen, da für die Kollisionsvermeidung stets der nächstliegende Punkt maßgeblich ist. Zwei Filterstufen sichern die Datenqualität: Messungen mit einer Signalqualität von null sowie Distanzen außerhalb des plausiblen Bereichs von 0,15 m bis 12 m werden verworfen. Nicht belegte Winkelklassen werden als unendlich mar-

kiert und damit explizit als Fehlmessung ohne Treffer gekennzeichnet. Diese Konvention setzt die Erkenntnis aus der Datenanalyse um, dass nicht-finite Werte ein regulärer Bestandteil von LiDAR-Messungen sind und definiert behandelt werden müssen. Das Ergebnis wird mit den vollständigen Metadaten des Nachrichtenformats, also Winkelbereich, Winkelinkrement, Messbereich und Scandauer, mit einer Rate von etwa 5,5 Umdrehungen pro Sekunde publiziert.

8.1.4 Stabilisierung der seriellen LiDAR-Kommunikation

Die Inbetriebnahme des realen LiDAR-Sensors scheiterte zunächst wiederholt: Der Verbindungsaufbau brach mit Protokollfehlern, Parse-Fehlern und scheinbaren Zeitüberschreitungen ab, obwohl der Sensor physisch einwandfrei arbeitete. Das Fehlerbild war nicht deterministisch reproduzierbar, was die Diagnose erheblich erschwerte. Die schrittweise Diagnose anhand der protokollierten Laufdaten ergab drei sich überlagernde Ursachen. Erstens verblieben nach abgebrochenen Scans Restbytes im Empfangspuffer der seriellen Schnittstelle, die bei der nächsten Anfrage als Antwortdaten fehlinterpretiert wurden und das Protokoll desynchronisierten. Zweitens schaltet das Öffnen des seriellen Ports über die DTR-Steuerleitung den Motor des Sensors ungewollt ein, wodurch bereits vor dem ersten Kommando Scandaten eintrafen. Drittens war der Standard-Timeout der Treiberbibliothek von einer Sekunde für einzelne Kommandos des Sensors zu knapp bemessen, sodass fälschlich Zeitüberschreitungen gemeldet wurden.

Die Lösung kombiniert mehrere Maßnahmen zu einem definierten, synchronisierten Verbindungsaufbau, den Abbildung 6 zeigt. Der Empfangspuffer wird an fünf festgelegten Protokollpunkten geleert: unmittelbar nach dem Öffnen des Ports, nach dem kontrollierten Stoppen von Scan und Motor mit anschließender Auslaufwartezeit, nach der Geräteabfrage, nach der Gesundheitsabfrage sowie unmittelbar vor dem Start des Scanbetriebs. Die Abfragen der Geräteinformationen über `get_info()` und des Gesundheitszustands über `get_health()` dienen dabei bewusst als Synchronisationspunkte, da sie aus einfachen, eindeutig zuordenbaren Anfrage-Antwort-Paaren bestehen. Der Timeout wurde auf drei Sekunden erhöht, und für Motorstart und Motorauslauf wurden definierte Wartezeiten eingeführt. Da verschiedene Versionen der Treiberbibliothek das interne Objekt der seriellen Schnittstelle unter unterschiedlichen Attributnamen führen,

wurde der Zugriff versionstolerant über eine Liste bekannter Attributnamen implementiert. Mit diesen Maßnahmen wurde der Verbindungsaufbau vollständig stabil und reproduzierbar. Tritt in einem beliebigen Schritt dennoch ein Fehler auf, wird die Verbindung getrennt und der Aufbau nach drei Sekunden Wartezeit wieder von Schritt 1 an begonnen. Dieser Pfad ist in Abbildung 6 rot gestrichelt von unten nach oben eingezeichnet.

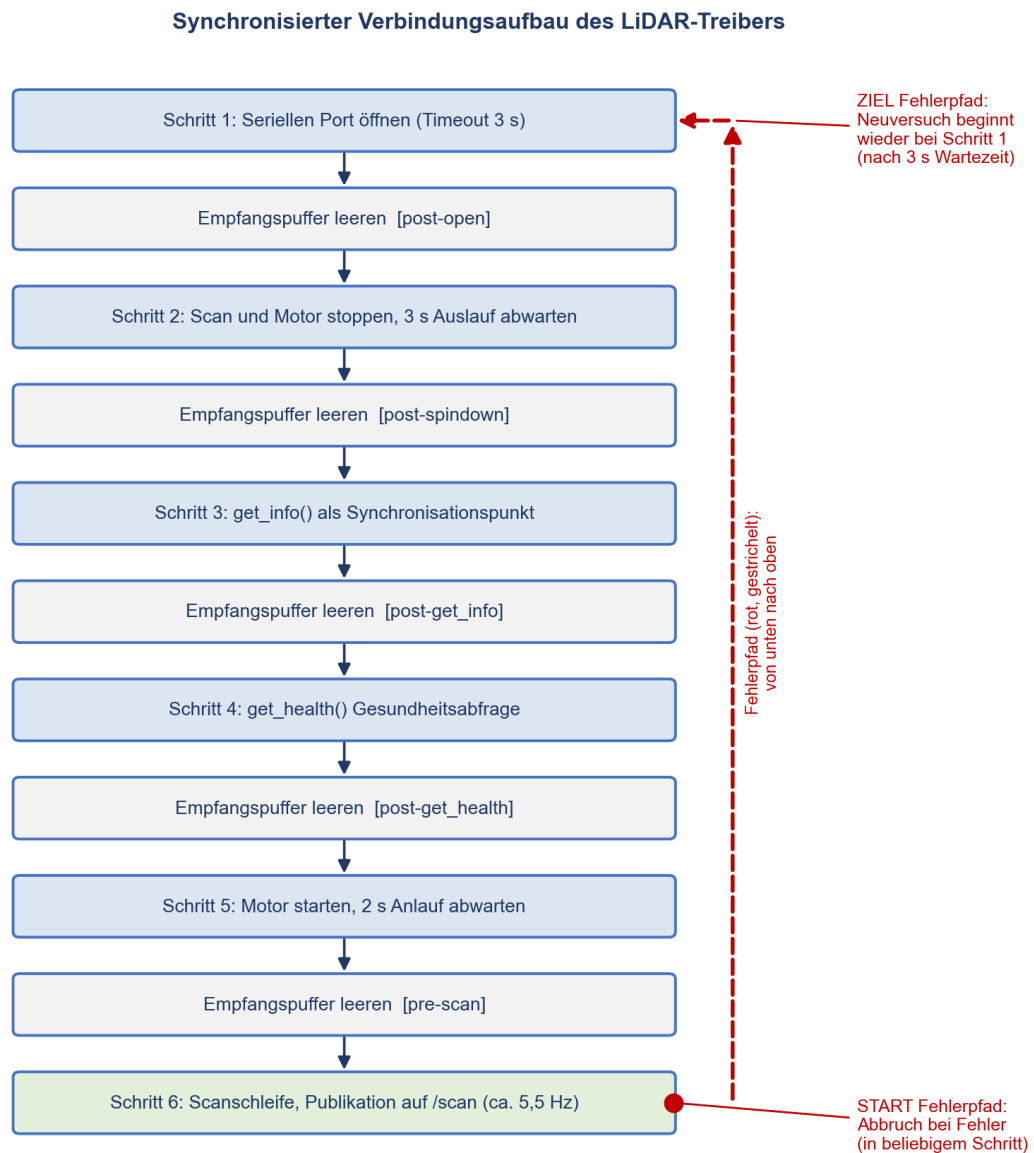


Abbildung 6: Synchronisierter Verbindungsaufbau des LiDAR-Treibers mit Flush-Strategie. Der rot gestrichelte Fehlerpfad führt vom Abbruchpunkt zurück zu Schritt 1

8.1.5 Sicherheitsgerichtete Auswertung: Not-Halt-Funktion

Für den manuellen Fahrbetrieb wurde eine eigenständige Sicherheitsschicht entworfen, der Knoten `safety_stop`. Er ist als Zwischeninstanz zwischen die WebSocket-Bridge und den Fahrzeugcontroller geschaltet: Die Bridge publiziert Nutzerbefehle auf dem Topic `vehicle_command_raw`, der Sicherheitsknoten reicht sie geprüft auf `vehicle_command` an den Controller weiter. Vorwärtsfahrbefehle passieren die Schicht nur dann unverändert, wenn der Frontsektor des Laserscans von plus minus 30 Grad um die Fahrzeuglängsachse frei ist. Wird ein Hindernis innerhalb von 1,0 m erkannt, begrenzt der Knoten die Geschwindigkeit auf null. Abbildung 7 fasst das Zustandsverhalten zusammen.

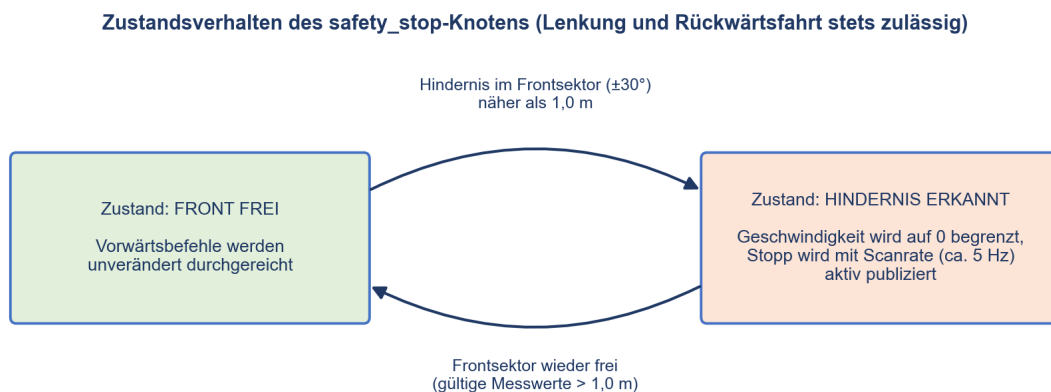


Abbildung 7: Zustandsverhalten der Not-Halt-Funktion (`safety_stop`)

Ein wesentliches Entwurfsdetail betrifft das Zeitverhalten. In der ersten Fassung wurde der Stoppbefehl erst mit dem nächsten eintreffenden Nutzerbefehl wirksam. Die Reaktionszeit hing damit von der Sendefrequenz der App ab. In der überarbeiteten Fassung publiziert der Knoten den Stoppbefehl direkt aus der Verarbeitungsroutine des Laserscans, also mit der Scanrate des Sensors von etwa 5 Hz und unabhängig von Nutzereingaben. Der Lenkwinkel bleibt dabei erhalten und Rückwärtsfahrt bleibt zulässig, sodass das Fahrzeug aus der Gefahrensituation herausmanövriert werden kann. Die Funktion ist damit fehlersicher ausgelegt, ohne den Bediener vollständig zu entmündigen.

8.1.6 Heatmap-basierte Visualisierung der LiDAR-Daten

Für die Bewertung der Sensordaten und die Kommunikation der Ergebnisse wurde eine Visualisierung der LiDAR-Daten nach dem Heatmap-Prinzip umgesetzt, bei der räumliche Messinformationen flächig und farb- beziehungsweise dichtekodiert dargestellt werden. Zwei komplementäre Darstellungswege wurden realisiert.

Der erste Weg dient der Offline-Analyse in Python. Aufgezeichnete Scans werden aus JSON-Dateien geladen, aus den Metadaten, also minimalem Winkel, maximalem Winkel und Winkelinkrement, werden die Messwinkel rekonstruiert und mit den zugehörigen Distanzen in einem DataFrame zusammengeführt. Die Darstellung erfolgt als Polardiagramm, in dem jeder Messpunkt entsprechend seiner Winkel- und Distanzkoordinate um den Sensormittelpunkt aufgetragen wird. Die Punktdichte der Darstellung wirkt dabei wie eine Intensitätskarte der Umgebung: Zusammenhängende Objektkonturen, etwa Wände oder Hindernisse, treten als dichte Punktzüge hervor, während Messausfälle als Lücken unmittelbar sichtbar werden. Diese Darstellung war das zentrale Werkzeug, um die Qualität der Datenbasis zu beurteilen, die Orientierung des Sensors zu verifizieren und die Filteranforderungen abzuleiten.

Der zweite Weg dient der Darstellung im Frontend. Die Flutter-App stellt ein Belegungsgitter (Occupancy Grid) [7] als zweidimensionales Zellenraster dar, in dem jede Zelle einen Belegungswert trägt und entsprechend eingefärbt wird. Diese Rasterdarstellung entspricht methodisch einer Heatmap der Umgebungsbelegung: Freie, belegte und unbekannte Bereiche sind auf einen Blick unterscheidbar. Die Datenstruktur des Gitters, bestehend aus Metadaten mit Auflösung, Breite, Höhe und Ursprungspose sowie einem flachen Datenfeld der Zellwerte, ist kompatibel zum ROS-Nachrichtentyp für Belegungsgitter aufgebaut. Damit ist die Darstellungsschicht bereits darauf vorbereitet, in einer späteren Ausbaustufe live berechnete Karten anstelle statischer Beispieldaten anzuzeigen.

8.1.7 Verarbeitung der Laufdaten

Unter Laufdaten werden hier sämtliche während des Betriebs und der Testläufe anfallenden Daten verstanden: aufgezeichnete Sensor-Scans einerseits und die Protokollausgaben der laufenden Knoten andererseits. Beide Datenarten wurden systematisch ausgewertet und flossen unmittelbar in Entwurfsentscheidungen ein.

Die neun im Projekt vorliegenden Scan-Aufzeichnungen wurden vollständig statistisch ausgewertet. Tabelle 7 fasst die Kennwerte zusammen. Aus dieser Auswertung wurden zwei Konsequenzen für die Implementierung abgeleitet: Erstens müssen nicht-finite Werte in jeder Verarbeitungsstufe als regulärer Fall behandelt werden, was in der Sektoraleswertung der Sicherheitsfunktion durch explizite Filterung umgesetzt ist. Zweitens bestätigte die Stabilität der Kennwerte über alle Aufzeichnungen die grundsätzliche Eignung des Sensors für die Hinderniserkennung im vorgesehenen Einsatzbereich.

Kennwert	Wert über alle neun Aufzeichnungen
Messpunkte je Scan	1147
Winkelabdeckung	Vollkreis, etwa $-\pi$ bis $+\pi$
Winkelauflösung	ca. 0,00548 rad (etwa 0,314 Grad)
Distanzbereich	ca. 0,47 m bis 10,4 m
Nicht-finite Werte je Scan	268 bis 278 (Messausfälle, außer Reichweite)
Scanrate des Sensors	ca. 5,5 Umdrehungen pro Sekunde

Tabelle 7: Statistische Kennwerte der aufgezeichneten LiDAR-Scans

Für die Auswertbarkeit der Laufdaten im Fahrbetrieb wurde ein abgestuftes Protokollierungskonzept umgesetzt. Die Vergabe der Log-Ebenen richtet sich nach dem diagnostischen Wert der Information. Tabelle 8 zeigt die Zuordnung. Auf diese Weise bleiben die Logs eines Testlaufs les- und auswertbar, ohne dass relevante Ereignisse im Rauschen der Bewegungsdaten untergehen.

Ereignis	Ebene	Begründung
Einzelne Bewegungskommandos (Fahren, Lenken)	DEBUG	Hochfrequent, im Normalbetrieb ohne Erkenntnisgewinn
Verbindungsaufbau des LiDAR (Geräteinfo, Gesundheitsstatus)	INFO	Dokumentiert den Systemzustand je Testlauf
Zeitüberschreitungen des Ultraschallsensors	WARN	Sicherheitsrelevant, deutet auf Hardware- oder Verdrahtungsprobleme hin
Hinderniserkennung der Not-Halt-Funktion (mit Distanz)	WARN	Sicherheitsrelevantes Ereignis, muss im Log sofort auffindbar sein
Puffer-Bereinigungen mit Byte-Anzahl je Protokollpunkt	DEBUG	Diagnosedaten für die Fehlersuche am seriellen Protokoll

Tabelle 8: Abgestuftes Protokollierungskonzept der Knoten

Darüber hinaus war die logbasierte Diagnose das zentrale Hilfsmittel bei der Inbetriebnahme des LiDAR-Sensors. Der Treiberknoten protokolliert an jedem Schritt des Ver-

bindungsaufbaus definierte Kenngrößen: die Anzahl der aus dem Empfangspuffer entfernten Bytes an jedem Bereinigungspunkt, die Geräteinformationen zu Modell, Firmware und Hardware sowie den Ergebniszustand der Gesundheitsabfrage des Sensors. Erst die systematische Auswertung dieser Laufdaten über mehrere Testläufe hinweg machte die Ursache der anfänglichen Verbindungsabbrüche identifizierbar.

8.2 Hinderniserkennung

In einem ersten Ansatz zur Hinderniserkennung werden die vom LiDAR-Sensor bereitgestellten Messdaten direkt ausgewertet. Hierzu werden sämtliche Messpunkte innerhalb eines festgelegten Maximalabstands betrachtet. Befindet sich ein Messwert innerhalb dieses Abstandes, wird dieser unmittelbar als Hindernis interpretiert. Dieses Verfahren ermöglicht zwar grundsätzlich die Erkennung von Objekten vor dem Fahrzeug, erweist sich im praktischen Betrieb jedoch als wenig robust.

Insbesondere führen einzelne fehlerhafte Messpunkte bereits zu einer Hinderniserkennung. Solche Ausreißer entstehen beispielsweise durch Reflexionen, Messrauschen oder ungültige Distanzwerte. Das Fahrzeug beginnt dadurch teilweise ein Ausweichmanöver, obwohl sich kein tatsächliches Hindernis vor ihm befindet. Zusätzlich werden auch Objekte berücksichtigt, die sich außerhalb des relevanten Fahrbereichs befinden und somit keinen Einfluss auf die aktuelle Fahrspur haben.

Zur Verbesserung der Hinderniserkennung können mehrere Maßnahmen umgesetzt werden. Zunächst wird der ausgewertete Sichtbereich des LiDAR-Sensors auf einen Winkelbereich vor dem Fahrzeug begrenzt. Dadurch werden ausschließlich Messpunkte innerhalb des für die Fahrtrichtung relevanten Winkels von -45° bis $+45^\circ$ berücksichtigt. Messpunkte außerhalb dieses Bereichs befinden sich seitlich oder hinter dem Fahrzeug und besitzen für die aktuelle Fahrentscheidung keine Relevanz. Gleichzeitig wird ein maximaler Erkennungsabstand eingeführt, sodass weit entfernte Objekte die Fahrentscheidung nicht beeinflussen.

Die Hinderniserkennung basiert auf der Auswertung der LiDAR-Messungen im vorderen Sichtbereich des Fahrzeugs. Da für das Fahrzeug in erster Linie das unmittelbar bevorstehende Hindernis relevant ist, wird aus allen gültigen Messpunkten der nächstgelegene ausgewählt. Dieser stellt das größte Kollisionsrisiko dar und bildet daher die Grundlage für die Ausweichplanung.

Zusätzlich werden ungültige LiDAR-Messwerte verworfen. Hierzu zählen insbesondere unendliche oder außerhalb des gültigen Messbereichs liegende Werte. Erst nach dieser Vorverarbeitung werden die verbleibenden Messpunkte zur Hinderniserkennung verwendet. Aus allen gültigen Messpunkten innerhalb des definierten Sichtfeldes und der maximalen Erkennungsdistanz wird anschließend der nächstgelegene Messpunkt als Hindernis ausgewählt. Die Kombination aus Sichtfeldbegrenzung, Distanzfilterung und der Validierung der Messwerte bildet somit die Grundlage der implementierten Hinderniserkennung.

Die aus der Hinderniserkennung ermittelte Position des nächstgelegenen Hindernisses wird anschließend an den Ausweichalgorithmus übergeben, welcher daraus einen kollisionsfreien Fahrpfad berechnet.

8.3 Gauß-basierte Hindernisvermeidung

Nach der Erkennung eines Hindernisses besteht die Aufgabe, dieses Hindernis zu vermeiden. Dazu soll ein geeigneter Fahrpfad berechnet und daraus ein Lenkwinkel für das Fahrzeug abgeleitet werden. Statt eines konstanten Lenkwinkels soll hierfür eine Gaußfunktion verwendet werden, um ein ruhiges Fahrverhalten zu erzeugen und nach dem Ausweichmanöver wieder auf die Ursprüngliche Spur zurück zu gelangen. Siehe hierzu 5.1.

Zu Beginn eines Ausweichmanövers speichert der Controller die aktuelle Fahrzeugposition als Referenzspur (`reference_y`). Diese Referenz bildet den Ausgangspunkt für die Berechnung der Ausweichspur. Anschließend wird die Position des Hindernisses aus den vom LiDAR gelieferten Winkel- und Entfernungsinformationen in kartesische Koordinaten umgerechnet. Auf Grundlage dieser Hindernisposition wird eine Gaußfunktion erzeugt, deren Amplitude aus dem erforderlichen seitlichen Sicherheitsabstand einschließlich der geschätzten Hindernisbreite berechnet wird. Die Gaußfunktion wird anschließend auf den Referenzpunkt des Fahrzeuges addiert und ergibt so eine kontinuierliche Sollspur.

Im Gegensatz zu einer vollständigen Spurplanung wird diese Ausweichspur jedoch nicht als fester Fahrplan bis zum Ende abgefahren. Stattdessen dient sie als lokale Orientierung für die Fahrzeugregelung. Während der Fahrt wird fortlaufend ein Lookahead-Punkt auf der berechneten Spur bestimmt. Aus der seitlichen Abweichung zwischen

Fahrzeugposition und diesem Zielpunkt wird der aktuell benötigte Lenkwinkel berechnet. Dadurch reagiert das Fahrzeug kontinuierlich auf den berechneten Fahrpfad, ohne für jeden Abschnitt einen festen Lenkwinkel vorzugeben.

Die Wahl einer Gaußfunktion bietet gegenüber einer konstanten Lenkstrategie mehrere Vorteile. Durch ihren stetigen Verlauf entstehen gleichmäßige Änderungen des Lenkwinkels, wodurch abrupte Richtungswechsel vermieden werden. Gleichzeitig kann die seitliche Auslenkung über die Parameter der Gaußfunktion an unterschiedliche Hindernisgrößen angepasst werden. Die Ausweichspur ist lokal begrenzt und beeinflusst daher nur den Bereich um das erkannte Hindernis.

Die aktuelle Implementierung stellt einen ersten funktionsfähigen Ansatz für eine Gaußbasierte Hindernisvermeidung dar. Bereits umgesetzt sind die Berechnung einer Gaußkurve, die Bestimmung eines Lenkwinkels anhand eines Lookahead-Punktes sowie die Speicherung der ursprünglichen Fahrspur als Referenz. Eine vollständige Spurverfolgung bis zur automatischen Rückkehr auf die Ausgangsspur ist jedoch noch nicht vollständig realisiert. Da der Controller in der aktuellen ROS-Node unmittelbar zurückgesetzt wird, sobald kein Hindernis mehr erkannt wird, wird die berechnete Ausweichspur derzeit nicht bis zu ihrem Ende verfolgt. Die bereits implementierte Rückkehrlogik im Controller wird dadurch aktuell nicht genutzt. Die vorhandene Referenzspur bildet jedoch bereits die Grundlage für eine spätere Erweiterung, bei der das Fahrzeug nach dem Umfahren eines Hindernisses selbstständig auf seine ursprüngliche Fahrbahn zurückkehrt.

Damit bildet die aktuelle Implementierung die Grundlage für eine kontinuierliche Hindernisvermeidung. Gegenüber einer einfachen festen Lenkstrategie ermöglicht sie bereits ein deutlich ruhigeres Fahrverhalten und schafft gleichzeitig die Voraussetzungen für zukünftige Erweiterungen, wie eine vollständige Spurverfolgung oder die Berücksichtigung mehrerer gleichzeitig erkannter Hindernisse.

Der vollständige Ablauf der implementierten Hindernisvermeidung ist in Abbildung 8 dargestellt.

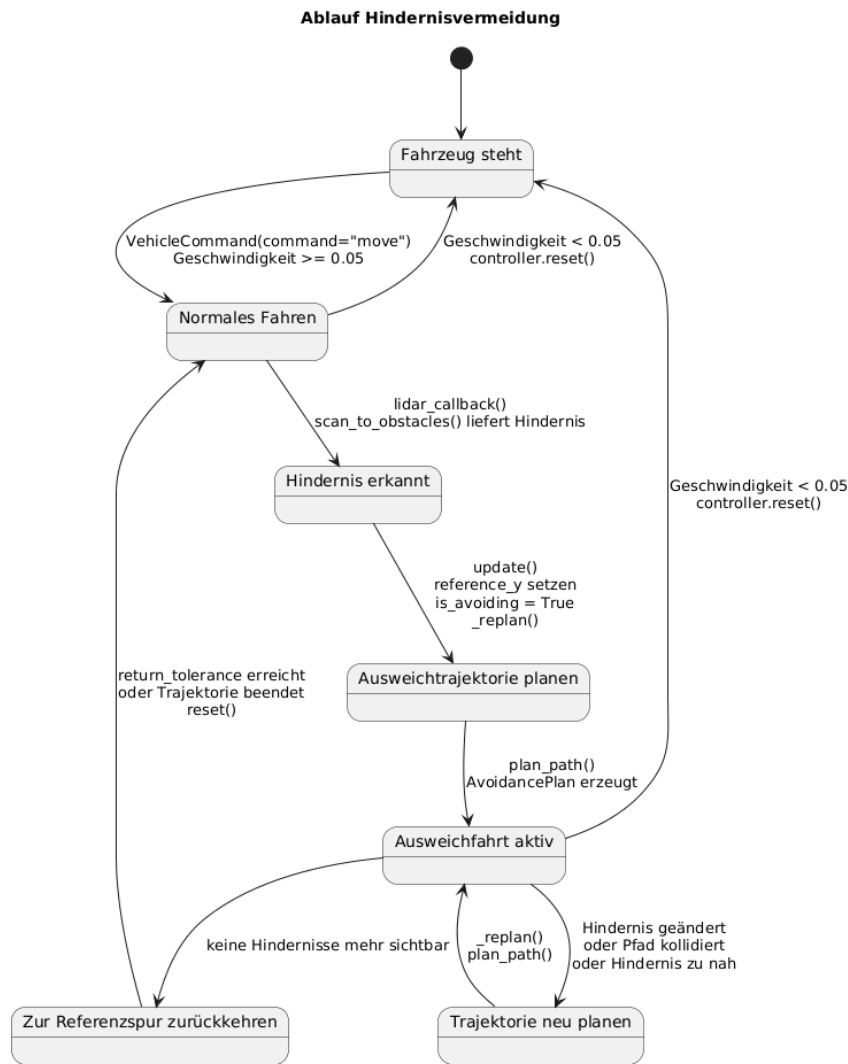


Abbildung 8: Zustandsdiagramm: Ausweichen

8.4 Optimierung der Spurplanung

Während der ersten Implementierung wurde die Ausweichspur nach jedem eingehenden LiDAR-Scan vollständig neu berechnet. Da sich die gemessenen Hindernispositionen durch Messrauschen geringfügig ändern, entstanden kontinuierlich unterschiedliche Spuren. Dieses permanente Neuplanen führte dazu, dass sich der berechnete Lenkwinkel ständig änderte.

Zur Reduzierung unnötiger Neuplanungen wird die berechnete Ausweichspur im Controller als aktive Spur gespeichert. Solange keine der definierten Neuplanungsbedingungen erfüllt ist, wird dieser Pfad weiterverwendet. Dabei werden seine Längskoordinaten in jedem Berechnungszyklus entsprechend der angenommenen Fahrzeugbewegung verschoben, sodass der Fortschritt entlang der Spur simuliert wird.

Eine erneute Berechnung erfolgt, wenn sich die gerundeten Hindernisdaten gegenüber der zu Beginn des Ausweichmanövers gespeicherten Situation verändern, wenn die aktive Spur den erforderlichen Sicherheitsabstand zu einem sichtbaren Hindernis unterschreitet oder wenn sich das Hindernis bereits innerhalb des definierten Sicherheitsbereichs befindet. Zur Unterdrückung kleiner Messschwankungen werden Hinderniswinkel auf ganze Grad, Entfernungen auf eine Nachkommastelle und die angenommene Winkelbreite ebenfalls auf ganze Grad gerundet.

In der aktuellen Implementierung wird durch die LiDAR-Auswertung jedoch ausschließlich der nächstgelegene gültige Messpunkt als Hindernis an den Controller übergeben. Die Erkennung mehrerer gleichzeitig vorhandener Hindernisse und damit die explizite Reaktion auf ein zusätzlich auftretendes Objekt ist derzeit nicht umgesetzt.

Zusätzlich wird der berechnete Lenkwinkel mittels exponentieller Glättung gefiltert. Hierbei wird der aktuelle Lenkwinkel nicht vollständig übernommen, sondern mit dem zuvor berechneten Wert kombiniert. Für den verwendeten Glättungsfaktor $\alpha = 0,7$ ergibt sich ein Tiefpassverhalten, das schnelle Änderungen unterdrückt und kleine Schwankungen der LiDAR-Messung ausgleicht. Dadurch entstehen weichere Lenkbewegungen und ein deutlich ruhigeres Fahrverhalten. Die aktuelle Implementierung plant nur bei relevanten Änderungen der Hindernissituation neu und glättet den Lenkwinkel mittels exponentieller Mittelung.

8.5 Priorisierung der Fahrbefehle

Während des Betriebs sendet die Smartphone-Anwendung kontinuierlich Fahrbefehle an den Fahrzeugcontroller. Gleichzeitig erzeugt die Hindernisvermeidung eigene Steuerbefehle, sobald ein Hindernis erkannt wird. Ohne zusätzliche Priorisierung würden beide Komponenten gleichzeitig Steuerbefehle senden.

In ersten Versuchen führte dies dazu, dass normale Fahrbefehle die berechneten Ausweichkommandos unmittelbar überschrieben. Das Fahrzeug begann zwar mit einem Ausweichmanöver, erhielt jedoch im nächsten Steuerzyklus erneut einen manuellen Fahrbefehl und kehrte sofort auf die ursprüngliche Fahrtrichtung zurück.

Zur Lösung dieses Problems besitzt der Befehl avoid innerhalb des Fahrzeugcontrollers eine höhere Priorität als normale Fahrbefehle (move). Während eines aktiven Ausweichmanövers werden eingehende Fahrbefehle der Smartphone-Anwendung igno-

riert. Erst nach erfolgreichem Abschluss der Trajektorie wird wieder auf die manuellen Steuerbefehle reagiert. Diese Priorisierung sorgt dafür, dass ein begonnenes Ausweichmanöver zuverlässig abgeschlossen wird. Die Prioritätslogik wird im Car-Controller über einen Zeitstempel realisiert, der normale move-Kommandos während eines aktiven avoid-Manövers unterdrückt.

Abbildung 9 zeigt abschließend die wesentlichen Klassen sowie deren Schnittstellen innerhalb der implementierten Hindernisvermeidung.

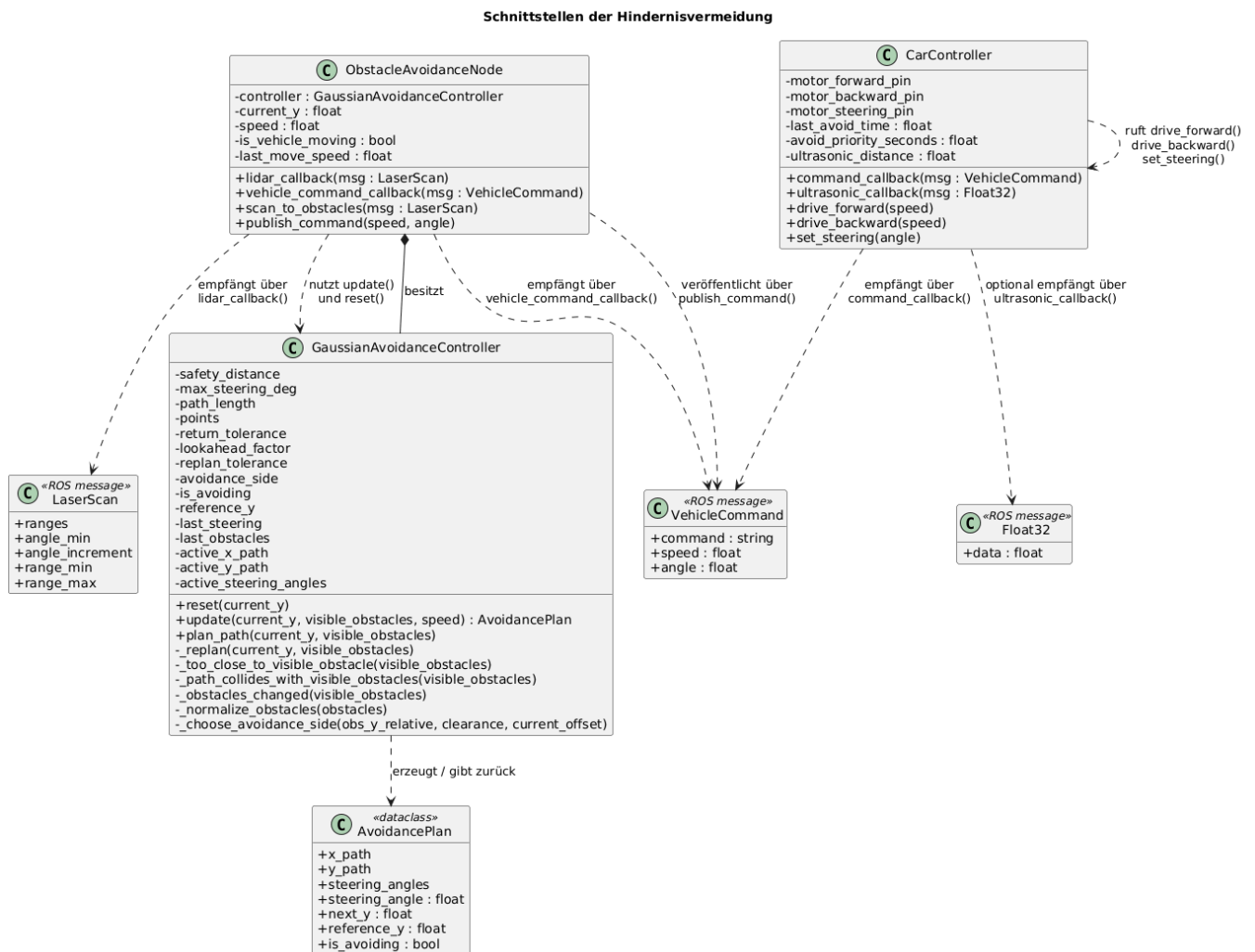


Abbildung 9: Klassendiagramm: Ausweichen

9 Aktualisierung der Applikation

Um die Neigungssteuerung funktionsfähig und zuverlässiger zu gestalten, wurde die Sensoranbindung überarbeitet. Die ursprünglich verwendete Bibliothek `motion_sensors` wurde durch die modernere Bibliothek `sensors_plus` ersetzt. Dadurch werden nicht mehr die bereits berechneten Pitch- und Roll-Werte ausgegeben, sondern die Rohdaten des Sensors, aus denen die Neigungswinkel Pitch und Roll mathematisch berechnet werden. Dadurch ist die Steuerung nicht mehr von der fehleranfälligen Orientierungsfunktion der ursprünglichen Bibliothek abhängig und kann auf unterschiedlichen Geräten zuverlässiger arbeiten.

Die ermittelten Neigungswerte werden anschließend nicht mehr unmittelbar an das Fahrzeug übertragen, sondern zunächst verarbeitet. Hierfür wurde eine Deadzone eingeführt, die sehr kleine Bewegungen des Smartphones ignoriert. Dadurch werden unbeabsichtigte Eingaben verhindert und das Fahrzeug bleibt bei ruhiger Haltung des Geräts stehen.

Zusätzlich wurde ein Low-Pass-Filter implementiert, der die Sensordaten glättet. Da Messwerte von Bewegungssensoren häufig Schwankungen und Rauschen enthalten, sorgt die Glättung für eine ruhigere und gleichmäßigere Fahrzeugsteuerung. Plötzliche kleine Ausschläge wirken sich dadurch nicht mehr unmittelbar auf das Fahrverhalten aus.

Um die Steuerung an unterschiedliche Fahrzeuge oder Nutzer anpassen zu können, wurden außerdem Verstärkungsfaktoren (Gain) für Geschwindigkeit und Lenkung eingeführt. Diese ermöglichen eine einfache Anpassung der Empfindlichkeit, ohne die eigentliche Berechnung der Sensordaten verändern zu müssen. Ergänzend wurden Optionen zum Invertieren der Achsen integriert, sodass sich die Steuerungsrichtung bei Bedarf anpassen lässt.

Die berechneten Steuerwerte werden abschließend auf einen zulässigen Wertebereich begrenzt, bevor sie über den `WebSocketManager` an das Fahrzeug übertragen werden. Dadurch wird sichergestellt, dass nur gültige Werte an das Fahrzeug gesendet werden und unerwartete Sensorausschläge keine unzulässigen Steuerbefehle erzeugen.

Neben der eigentlichen Sensorverarbeitung wurden auch die Einstellungen innerhalb der App angepasst. Über einen Button kann nun ausgewählt werden, ob die Steuerung per Neigung oder per Joystick erfolgen soll. Die Neigungssteuerung wird nur dann ge-

laden, wenn sie explizit aktiviert wurde. Dadurch unterstützt die Anwendung mehrere Steuerungsarten und bietet dem Nutzer mehr Flexibilität.

Darüber hinaus wurde die Verwaltung des Sensor-Streams verbessert. Beim Verlassen der Seite wird die Verbindung zum Sensor ordnungsgemäß beendet, wodurch unnötiger Ressourcenverbrauch und mögliche Fehler durch weiterhin laufende Sensorabfragen vermieden werden. Insgesamt führen diese Änderungen zu einer stabileren, besser anpassbaren und deutlich präziseren Neigungssteuerung.

10 Ergebnisse

Zur Evaluation des entwickelten Systems wurden die in Kapitel 5 definierten Testszenarien jeweils zehnmal durchgeführt. Bewertet wurden die Erfolgsquoten der Hinderniserkennung und des Ausweichmanövers.

Kennzahl	Szenario A	Szenario B	Szenario C
Erfolgsquote Hinderniserkennung	100 %	100 %	95 %
Erfolgsquote Ausweichmanöver	100 %	90 %	80 %

Tabelle 9: Evaluation der Hinderniserkennung und Ausweichfunktion

Die Hinderniserkennung arbeitete in allen Testszenarien zuverlässig. Lediglich bei zwei aufeinanderfolgenden Hindernissen kam es in einem Versuch zu einer unvollständigen Erfassung der Umgebung, wodurch die Erkennungsrate auf 95 % sank. Die höchste Erfolgsquote beim Ausweichen wurde im Szenario mit einem zentral auf der Fahrbahn platzierten Hindernis erreicht. Bei komplexeren Hindernissituationen nahm die Erfolgsquote erwartungsgemäß ab, da die aktuelle Implementierung stets nur das nächstgelegene Hindernis in die Spurplanung einbezieht.

10.1 Qualitative Bewertung

Im Rahmen dieser Arbeit wurde ein erster funktionsfähiger Ansatz zum autonome Ausweichen von Hindernissen entwickelt und erfolgreich in das Fahrzeug integriert. Das System ist in der Lage, die Messdaten des LiDAR-Sensors auszuwerten, Hindernisse im vorderen Sichtbereich zu erkennen und daraus automatisch geeignete Fahrbefehle abzuleiten.

Die implementierte Hinderniserkennung berücksichtigt ausschließlich gültige Messwerte innerhalb eines definierten Sichtfeldes und eines maximalen Erkennungsabstands. Aus allen erkannten Messpunkten wird das nächstgelegene Hindernis ausgewählt und als Grundlage für die Fahrwegplanung verwendet. Dadurch werden viele fehlerhafte Messungen und für die Fahrtrichtung irrelevante Objekte bereits im Vorfeld ausgefiltert.

Für die Hindernisvermeidung wurde ein Gauß-basierter Ansatz umgesetzt. Aus der Position des erkannten Hindernisses wird eine kontinuierliche Ausweichspur berechnet, entlang derer der benötigte Lenkwinkel mithilfe eines Lookahead-Punktes bestimmt

wird. Gegenüber einer einfachen konstanten Lenkstrategie führt dieser Ansatz zu deutlich ruhigeren und gleichmäßigeren Lenkbewegungen.

Durch die Einführung einer ereignisbasierten Neuplanung wird die Spur nicht mehr nach jedem LiDAR-Scan vollständig neu berechnet. Stattdessen erfolgt eine Neuberechnung nur bei relevanten Änderungen der Hindernissituation. Zusätzlich reduziert eine exponentielle Glättung des Lenkwinkels die Auswirkungen von Messrauschen und verhindert abrupte Richtungsänderungen. Dadurch konnte das Fahrverhalten gegenüber den ersten Implementierungsständen deutlich stabilisiert werden.

Weiterhin wurde eine Priorisierung der Fahrbefehle umgesetzt. Während eines aktiven Ausweichmanövers besitzen die automatisch erzeugten Ausweichbefehle eine höhere Priorität als die von der Smartphone-Anwendung gesendeten Fahrbefehle. Dadurch wird verhindert, dass ein begonnenes Ausweichmanöver durch manuelle Steuerbefehle unterbrochen wird.

Insgesamt zeigt die entwickelte Lösung, dass eine Gauß-basierte Spurplanung in Verbindung mit einer kontinuierlichen Lenkwinkelregelung eine robuste Grundlage für die autonome Hindernisvermeidung des Fahrzeugs bildet. Die bisherigen Ergebnisse ermöglichen bereits ein zuverlässiges Erkennen einzelner Hindernisse sowie deren ruhiges und kontrolliertes Umfahren.

10.2 Herausforderungen und Weiterentwicklungen

Die entwickelte Hindernisvermeidung stellt einen ersten funktionsfähigen Ansatz für das autonome Ausweichen dar. Im praktischen Betrieb haben sich jedoch mehrere Herausforderungen gezeigt, die im Rahmen zukünftiger Arbeiten weiter untersucht und verbessert werden sollten.

Eine wesentliche Einschränkung besteht darin, dass das System derzeit ausschließlich auf das Ausweichen von Hindernissen ausgelegt ist. Kann aufgrund der Umgebung oder des verfügbaren Platzes keine kollisionsfreie Spur berechnet werden, fährt das Fahrzeug weiterhin auf das Hindernis zu. Eine automatische Notbremsfunktion ist aktuell nicht implementiert. Zukünftig sollte daher geprüft werden, ob ein Ausweichmanöver überhaupt möglich ist. Ist dies nicht der Fall, muss das Fahrzeug rechtzeitig abbremsen und anhalten, um eine Kollision zu vermeiden.

Eine weitere Herausforderung betrifft die Rückkehr auf die ursprüngliche Fahrspur.

Zwar speichert der Controller bereits die ursprüngliche Referenzspur und enthält Ansätze für eine automatische Rückführung, die berechnete Gaußkurve wird jedoch derzeit nicht vollständig bis zu ihrem Ende verfolgt. Dadurch setzt das Fahrzeug seine Fahrt nach dem Ausweichmanöver nicht automatisch auf der ursprünglichen Spur fort. Erste Versuche einer vollständigen Spurverfolgung führten insbesondere wegen ungenauen Lenkwinkeln oder mehreren Hindernissen in der Umgebung dazu, dass das Fahrzeug teilweise in Kreisbewegungen fuhr. Hier sind robustere Regelungsverfahren sowie eine zuverlässigere Spurverfolgung erforderlich.

Zudem traten vereinzelt Probleme mit dem LiDAR-Sensor auf. In einigen Fällen wurde der Sensor nach dem Start des Systems nicht korrekt initialisiert oder fiel während der Fahrt sporadisch aus. Die genaue Ursache konnte bislang nicht eindeutig bestimmt werden. Aufgrund der beobachteten Symptome werden jedoch Probleme in der Stromversorgung oder Spannungsstabilität vermutet. Eine Analyse der Hardwareversorgung sowie eine geeignete Fehlererkennung sollten Bestandteil zukünftiger Arbeiten sein, um die Zuverlässigkeit des Gesamtsystems zu erhöhen.

Die aktuelle Hinderniserkennung betrachtet jeweils den nächstgelegenen LiDAR-Messpunkt als einzelnes Hindernis. Für eine robustere Objekterkennung wäre eine Clusterung benachbarter Messpunkte sinnvoll. Dadurch könnten zusammenhängende Hindernisse als vollständige Objekte erkannt sowie deren Ausdehnung bestimmt werden. Insbesondere die geschätzte Hindernisbreite könnte anschließend direkt in die Berechnung der Gaußkurve einfließen und so eine realistischere Ausweichbewegung ermöglichen. Darüber hinaus besteht weiterer Optimierungsbedarf bei der Parametrisierung der Fahrzeugregelung. Der optimale Sicherheitsabstand sowie die benötigten Lenkwinkel sollten optimal eingestellt werden. Aufgrund des vergleichsweise kleinen maximalen Lenkwinkels und der langen Bremswege des Fahrzeugs muss eine Hinderniserkennung bereits in größerer Entfernung erfolgen. Gleichzeitig erfordern diese Fahrzeugeigenschaften großräumige Ausweichspuren, welche in engen Umgebungen wiederum das Risiko neuer Kollisionen erhöhen können. Eine adaptive Anpassung des Sicherheitsabstands und der Spurparameter an die jeweilige Fahrsituation stellt daher einen vielversprechenden Ansatz für zukünftige Entwicklungen dar.

11 Diskussion

Die durchgeführten Experimente zeigen, dass der gewählte Gauß-basierte Ansatz grundsätzlich für die lokale Hindernisvermeidung geeignet ist. Im Vergleich zu globalen Planungsverfahren wie A* oder SLAM-basierten Navigationslösungen besitzt der entwickelte Ansatz deutlich geringere Anforderungen an Rechenleistung und Speicherbedarf. Dadurch eignet er sich insbesondere für ressourcenbeschränkte Systeme wie den Raspberry Pi. Gleichzeitig ergeben sich Einschränkungen. Da ausschließlich lokale Sensordaten betrachtet werden, besitzt das Fahrzeug keine Kenntnis über die globale Umgebung. Komplexe Hindernisanordnungen können daher nicht optimal berücksichtigt werden. Die in den Testläufen erreichte mittlere Erfolgsquote von 90 % bei den Ausweichmanövern zeigt, dass der entwickelte Ansatz grundsätzlich für autonome Fahrsysteme im Modellmaßstab geeignet ist. Die Hinderniserkennung erreichte über alle Testszenarien hinweg eine Erfolgsquote von 98 %. Für einen produktiven Einsatz wären jedoch zusätzliche Verfahren zur Kartierung, Lokalisierung und globalen Pfadplanung erforderlich.

12 Fazit und Ausblick

Die Evaluation zeigte, dass das entwickelte System Hindernisse mit einer hohen Erfolgsquote erkennen und in den meisten Testläufen erfolgreich umfahren konnte. Damit konnte nachgewiesen werden, dass die Kombination aus LiDAR-basierter Hinderniserkennung, Gauß-basierter Trajektorienplanung und ROS 2-basierter Systemarchitektur eine praxistaugliche Grundlage für autonomes Fahren auf ressourcenbeschränkter Embedded-Hardware darstellt.

12.1 Ausblick und Weiterentwicklungsmöglichkeiten

Die implementierte LiDAR-Verarbeitungskette, die Sicherheitsfunktion, die Visualisierung und die Container-Infrastruktur bilden eine tragfähige Grundlage, eröffnen jedoch an mehreren Stellen konkrete und wissenschaftlich lohnende Weiterentwicklungspfade. Tabelle 10 ordnet die Pfade ein. Die folgenden Abschnitte beschreiben zu jedem Pfad, wie eine Umsetzung konkret erfolgen könnte, und begründen, warum gerade dieser Weg gewählt werden sollte.

Weiterentwicklungspfad	Baut auf auf	Erster konkreter Schritt
Zeitliche Filterung und Clusterbildung	Aufbereitungsstufe des Treibers	Ringpuffer über die letzten 3 bis 5 Scans, gleitender Median je Winkelklasse
Live-Heatmaps und Belegungsgitter	Topic /scan, Flutter-Rasteranzeige	Knoten mit Ray Casting (Bresenham) und Log-Odds-Aktualisierung
SLAM und Lokalisierung	Stabile LaserScan-Publikation	Integration von slam_toolbox, Aufbau des TF-Baums
Erweiterte Sicherheitsfunktion	safety_stop-Knoten	Geschwindigkeitsabhängige Auslösedistanz aus Bremsversuchen bestimmen
Automatisierte Laufdaten-Auswertung	Protokollierungskonzept	Aufzeichnung aller Topics mit rosbag2 je Testlauf
CI/CD für Container	Multi-Stage-Build	GitHub-Actions-Workflow mit docker buildx für ARM64

Tabelle 10: Weiterentwicklungspfade im Überblick

12.1.1 Weiterentwicklung der LiDAR-Datenverarbeitung

Die aktuelle Aufbereitung der Scandaten arbeitet scanweise und aggregiert je Winkelklasse die minimale Distanz. Drei Ausbaustufen bieten sich an, die jeweils eine konkrete Schwäche des aktuellen Stands adressieren.

Erstens eine zeitliche Filterung: Statt jeden Scan isoliert zu betrachten, könnten die letzten N Scans, beispielsweise N gleich 3 bis 5, je Winkelklasse zu einem gleitenden Median zusammengefasst werden. Die Begründung liegt im Messverhalten des Sensors: Einzelne Ausreißer, wie sie durch Reflexionen oder Messrauschen entstehen, treten typischerweise nur in einem einzelnen Scan auf, während echte Hindernisse über mehrere Scans stabil gemessen werden. Der Median wurde gegenüber dem Mittelwert bewusst vorgeschlagen, da er gegenüber einzelnen Extremwerten robust ist und ein einzelner Ausreißer das Ergebnis nicht verschiebt. Die Umsetzung erfordert lediglich einen Ringpuffer der letzten Scans im Treiber oder in einem eigenen Filterknoten sowie eine Medianbildung je Winkelklasse. Die zusätzliche Latenz von wenigen hundert Millisekunden bleibt für die gefahrenen Geschwindigkeiten unkritisch.

Zweitens eine Clusterbildung auf Treiberebene: Benachbarte Messpunkte ähnlicher Distanz könnten zu Objekthypothesen zusammengefasst werden, sodass nachgelagerte Knoten nicht mehr mit Rohpunkten, sondern mit Objekten samt Position, Ausdehnung und nächstem Punkt arbeiten. Ein einfacher Ansatz ist das sequentielle Durchlaufen der Winkelklassen mit einem Distanzsprung-Kriterium. Anspruchsvoller wäre ein dichte-basiertes Verfahren wie DBSCAN [23] auf den in kartesische Koordinaten transformierten Punkten. Dies würde die bereits in der Hinderniserkennung angelegte Clusterlogik systematisch verallgemeinern und allen Konsumenten zugutekommen lassen, statt sie in jedem Knoten erneut zu implementieren.

Drittens die Umstellung des Treibers auf einen etablierten, gepflegten ROS-Treiber, etwa das Paket `slidar_ros2` des Herstellers, oder alternativ eine Reimplementierung des eigenen Treibers in C++ als Lifecycle-Node. Lifecycle-Nodes erlauben einen deterministischen Startablauf über die Phasen Konfigurieren, Aktivieren und Deaktivieren und würden die bisher über Wartezeiten gelöste Startreihenfolge der Knoten sauber formalisieren. Der Erhalt des eigenen Treibers hätte demgegenüber den Vorteil der vollständigen Kontrolle über die entwickelte Synchronisationsstrategie. Welche Option vorzuziehen ist, sollte anhand eines Stabilitätsvergleichs beider Treiber über längere

Testläufe entschieden werden.

12.1.2 Live-Heatmaps und Umgebungskarten

Die vorhandene Heatmap-Darstellung visualisiert bislang statische Beispieldaten. Der naheliegende nächste Schritt ist die Erzeugung eines Belegungsgitters zur Laufzeit aus dem Topic `/scan`. Konkret würde ein neuer Knoten jeden Laserscan mittels Strahlverfolgung, beispielsweise über den Bresenham-Algorithmus, in ein Raster eintragen: Zellen entlang des Strahls gelten als frei, die Zelle am Messpunkt als belegt. Um Messrauschen zu glätten, sollte die Aktualisierung nicht binär, sondern probabilistisch über Log-Odds erfolgen, wie es in der Literatur zu Belegungsgittern üblich ist [7]. Die Begründung ist, dass eine einzelne fehlerhafte Messung eine Zelle dann nicht dauerhaft umkippen kann, sondern erst wiederholte übereinstimmende Messungen den Zellzustand festigen. Das Ergebnis ließe sich direkt als Standard-Nachricht `nav_msgs/OccupancyGrid` publizieren, wodurch etablierte Werkzeuge wie RViz ohne Zusatzaufwand zur Kontrolle nutzbar wären.

Für die Anzeige in der App müsste die WebSocket-Bridge um die Übertragung des Gitters ergänzt werden. Da vollständige Gitter bei hoher Auflösung groß werden, bieten sich zwei Optimierungen an: die Übertragung nur geänderter Zellbereiche (Delta-Kodierung) sowie eine reduzierte Auflösung für die mobile Darstellung. Die bestehende Flutter-Rasterdarstellung könnte unverändert weiterverwendet werden, da ihre Datenstruktur bereits dem ROS-Nachrichtenformat nachempfunden ist. Genau für diesen Übergang wurde die Kompatibilität seinerzeit vorgesehen. Lediglich die Farbskala sollte von einer binären auf eine kontinuierliche Belegungswahrscheinlichkeit umgestellt werden.

Darüber hinaus eröffnen aggregierte Heatmaps über mehrere Testläufe hinweg eine analytische Perspektive: Werden die Belegungsgitter mehrerer Fahrten überlagert, entstehen Aufenthalts- und Hinderniskarten des Einsatzbereichs, aus denen sich systematische Problemzonen wie schlecht erfassbare Materialien oder tote Winkel ablesen lassen. Dies wäre ein unmittelbar publikationsfähiger Auswertungsansatz für Folgearbeiten.

12.1.3 Selbstlokalisierung und Kartierung (SLAM)

Mit der stabilen Publikation von Laserscans ist die zentrale Voraussetzung für simultane Lokalisierung und Kartierung [2] geschaffen. Ein konkreter Umsetzungspfad wäre die Integration des Pakets `slam_toolbox` [11], das in ROS 2 als Standard gilt und direkt auf `/scan` aufsetzt. Die Wahl dieses Pakets ist durch seine aktive Pflege, die Dokumentationslage und die direkte Kompatibilität mit dem vorhandenen Nachrichtenformat begründet. Dafür müssten zwei Lücken geschlossen werden: Erstens benötigt SLAM eine Odometrie-Schätzung. Diese könnte aus Radencodern gewonnen oder ersatzweise über eine inertielle Messeinheit gestützt werden, wobei sich für die Fusion das Paket `robot_localization` mit einem erweiterten Kalman-Filter [14] anbietet. Zweitens muss ein konsistenter Koordinatenrahmenbaum (TF-Baum) mit den Rahmen für Karte, Odometrie, Fahrzeugbasis und Sensor definiert werden. Die statische Transformation zwischen Fahrzeugbasis und Sensor ergibt sich aus der Einbauposition des LiDAR. Als Ergebnis stünde eine konsistente Umgebungskarte zur Verfügung, die sowohl die Heatmap-Anzeige der App speisen als auch als Grundlage für globale Pfadplanung dienen könnte.

12.1.4 Erweiterung der Sicherheitsfunktion

Die implementierte Not-Halt-Funktion arbeitet mit einem festen Abstandsschwellenwert von 1,0 m und einem festen Frontsektor. Drei Erweiterungen würden die Schutzwirkung deutlich erhöhen.

Erstens ein geschwindigkeitsabhängiger Anhalteweg: Statt eines festen Schwellenwerts sollte die Auslösedistanz aus der aktuellen Geschwindigkeit berechnet werden, näherungsweise als

$$d_{\text{stopp}} = v \cdot t_r + \frac{v^2}{2 a_{\text{max}}},$$

wobei t_r die Reaktionszeit des Systems aus Scanrate, Verarbeitungs- und Stellzeit und a_{max} die maximal erreichbare Verzögerung bezeichnet. Die Begründung liegt auf der Hand: Ein fester Schwellenwert ist bei niedriger Geschwindigkeit unnötig konservativ und bei hoher Geschwindigkeit unter Umständen zu knapp. Beide Größen ließen sich experimentell auf dem Fahrzeug bestimmen. Die Reaktionszeit folgt unmittelbar aus den bereits protokollierten Laufdaten, die Verzögerung aus Bremsversuchen mit Di-

stanzmessung.

Zweitens ein lenkwinkelabhängiger Überwachungssektor: Bei Kurvenfahrt liegt der gefährdete Bereich nicht symmetrisch vor dem Fahrzeug. Der überwachte Winkelbereich sollte daher mit dem aktuellen Lenkwinkel mitgeführt werden, sodass der Sektor der tatsächlichen Fahrschlauchrichtung folgt. Die dazu nötige Information liegt in der Sicherheitsschicht bereits vor, da sie sämtliche Fahrbefehle durchleitet. Die Erweiterung erfordert somit keine neue Schnittstelle.

Drittens Robustheitsmechanismen auf Systemebene: Eine Hysterese zwischen Auslöse- und Freigabedistanz würde das Flattern der Freigabe an der Schwelle verhindern. Ein Totmann-Mechanismus (Watchdog) sollte das Fahrzeug zusätzlich stoppen, wenn für eine definierte Zeit keine Laserscans mehr eintreffen, da die Schutzfunktion sonst bei Sensorausfall stillschweigend unwirksam würde. Dieser Fall ist besonders kritisch, weil er vom Bediener nicht bemerkt werden kann. Schließlich böte die Fusion mit dem Ultraschallsensor eine redundante zweite Messquelle im Nahbereich, deren Plausibilisierung gegen den LiDAR Fehlmessungen beider Sensoren erkennbar machen würde.

12.1.5 Systematisierung der Laufdatenverarbeitung

Die bislang manuell durchgeführte Auswertung der Laufdaten ließe sich zu einer automatisierten Auswertungskette ausbauen. Konkret bietet ROS 2 mit `rosbag2` ein Standardwerkzeug, um alle Topics eines Testlaufs, also Scans, Fahrbefehle und Sicherheitsereignisse, vollständig aufzuzeichnen. Darauf aufbauend könnte eine Auswertungs-pipeline in Python die Aufzeichnungen einlesen und je Lauf definierte Kennzahlen berechnen: erreichte Scanrate, Latenz zwischen Hinderniserkennung und Stoppbefehl, Häufigkeit ungültiger Messwerte, Anzahl und Dauer von Verbindungsabbrüchen sowie Auslösestatistik der Sicherheitsfunktion. Die Kennzahlen ließen sich als Zeitreihen ablegen und mit Werkzeugen wie Foxglove Studio oder einfachen Diagrammen visualisieren.

Besonders wertvoll wäre die Nutzung aufgezeichneter Läufe als Regressionstests: Aufgezeichnete Scans können in der Entwicklungsumgebung wieder abgespielt werden, sodass Änderungen an Filterung, Hinderniserkennung oder Sicherheitslogik reproduzierbar gegen identische Eingabedaten geprüft werden können, ohne dass das physische Fahrzeug benötigt wird. Die Begründung für diesen Ansatz liegt in der bereits

umgesetzten Trennung von Hardware und Logik: Da alle Konsumenten ausschließlich auf dem Topic `/scan` arbeiten, ist es für sie unerheblich, ob die Daten vom realen Sensor oder aus einer Aufzeichnung stammen. Damit würde diese Trennung konsequent bis in die Qualitätssicherung fortgeführt.

12.1.6 Automatisierung der Container-Infrastruktur

Der eingeführte Multi-Stage-Build wird bislang manuell angestoßen. Der folgerichtige Ausbau ist eine kontinuierliche Integrationskette, beispielsweise über GitHub Actions: Bei jeder Änderung würden die ROS-Pakete automatisch gebaut, die Container-Images über `docker buildx` nativ für die Zielarchitektur ARM64 erzeugt und in einer Registry abgelegt. Das Fahrzeug könnte fertige Images dann nur noch beziehen, statt sie lokal zu bauen, was die Inbetriebnahmezeit auf dem Raspberry Pi erheblich verkürzen würde. Das Bauen auf dem Pi selbst hat sich in den Testläufen als der zeitaufwendigste Einzelschritt erwiesen. Ergänzend sollten Gesundheitsprüfungen (Healthchecks) im Compose-Manifest definiert werden, damit der Zustand der Knoten von außen überwachbar ist, sowie getrennte Compose-Profile für Simulations- und Fahrzeugbetrieb, um die bestehenden Simulationsrückfälle explizit zu konfigurieren. Perspektivisch ließe sich der Ressourcenbedarf auf dem Zielsystem weiter senken, indem die Python-Knoten als komponierbare Knoten (Composable Nodes) in einem gemeinsamen Prozess zusammengeführt werden, wodurch der Kommunikationsaufwand zwischen den Knoten entfällt.

12.1.7 Zusammenfassende Einordnung

Die genannten Pfade bauen sämtlich unmittelbar auf den geschaffenen Grundlagen auf: Die zeitliche Filterung und die Clusterbildung erweitern die bestehende Aufbereitungsstufe, Live-Heatmaps und SLAM setzen auf dem stabilen Scan-Topic auf, die erweiterte Sicherheitsfunktion verfeinert die vorhandene Schutzschicht, und die Automatisierung der Infrastruktur führt den eingeschlagenen Weg der Reproduzierbarkeit konsequent fort. Die Arbeit hinterlässt damit nicht nur ein funktionsfähiges System, sondern ein anschlussfähiges Fundament für wissenschaftliche Folgearbeiten.

Literaturverzeichnis

- [1] Alasdair Allan, *Benchmarking Raspberry Pi 5*, Raspberry Pi, Okt. 2023. besucht am 7. Juni 2026.
- [2] C. Cadena u. a., „Past, Present, and Future of Simultaneous Localization and Mapping: Toward the Robust-Perception Age,“ *IEEE Transactions on Robotics*, Jg. 32, Nr. 6, S. 1309–1332, 2016. DOI: 10.1109/TR0.2016.2624754
- [3] Canonical Ltd, *Install Ubuntu on a Raspberry Pi*, Website. besucht am 7. Juni 2026.
- [4] Docker Inc., *Resource Constraints*. besucht am 7. Juni 2026.
- [5] Docker Inc., *Runtime Metrics*. besucht am 7. Juni 2026.
- [6] E. Gamess und M. Parajuli, „Performance Evaluation of the Docker Technology on Different Raspberry Pi Models,“ in *Proceedings of the 2023 5th International Electronics Communication Conference*, Osaka City Japan: ACM, Juli 2023, S. 27–37, ISBN: 979-8-4007-0885-5. DOI: 10.1145/3616480.3616485 besucht am 6. Juli 2026.
- [7] A. Hornung, K. M. Wurm, M. Bennewitz, C. Stachniss und W. Burgard, „Octo-Map: An Efficient Probabilistic 3D Mapping Framework Based on Octrees,“ *Autonomous Robots*, Jg. 34, Nr. 3, S. 189–206, 2013. DOI: 10.1007/s10514-012-9321-0
- [8] Institute of Electrical and Electronics Engineers und IEEE Computer Society, Hrsg., *2015 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS 2015): Philadelphia, Pennsylvania, USA, 29 - 31 March 2015*, An Updated Performance Comparison of Virtual Machines and Linux Containers. Piscataway, NJ: IEEE, 2015, ISBN: 978-1-4799-1957-4 978-1-4799-1956-7.
- [9] Y. Li und J. Ibanez-Guzman, „Lidar for Autonomous Driving: The Principles, Challenges, and Trends for Automotive Lidar and Perception Systems,“ *IEEE Signal Processing Magazine*, Jg. 37, Nr. 4, S. 50–61, 2020. DOI: 10.1109/MSP.2020.2973615

- [10] S. Macenski, T. Foote, B. Gerkey, C. Lalancette und W. Woodall, „Robot Operating System 2: Design, architecture, and uses in the wild,“ *Science Robotics*, Jg. 7, Nr. 66, eabm6074, 2022. DOI: 10.1126/scirobotics.abm6074
- [11] S. Macenski und I. Jambrecic, „SLAM Toolbox: SLAM for the dynamic world,“ *Journal of Open Source Software*, Jg. 6, Nr. 61, S. 2783, 2021. DOI: 10.21105/joss.02783
- [12] Y. Maruyama, S. Kato und T. Azumi, „Exploring the Performance of ROS2,“ in *Proceedings of the 13th International Conference on Embedded Software (EMSOFT)*, Pittsburgh, PA: ACM, 2016, S. 1–10. DOI: 10.1145/2968478.2968502
- [13] D. Merkel, „Docker: Lightweight Linux Containers for Consistent Development and Deployment,“ *Linux Journal*, Jg. 2014, Nr. 239, S. 2, 2014.
- [14] T. Moore und D. Stouch, „A Generalized Extended Kalman Filter Implementation for the Robot Operating System,“ in *Intelligent Autonomous Systems 13*, Cham: Springer International Publishing, 2016, S. 335–348. DOI: 10.1007/978-3-319-08338-4_25
- [15] Open Robotics, *ROS 2 on Raspberry Pi*, 2026. besucht am 7. Juni 2026.
- [16] pentiumsoak, *Operating Systems Supported by Raspberry Pi*, Sep. 2024. besucht am 7. Juni 2026.
- [17] Raspberry Pi Ltd, „Raspberry Pi 4 Model B,“ Raspberry Pi Ltd, Techn. Ber., Apr. 2026. besucht am 7. Juli 2026.
- [18] Raspberry Pi Ltd, „Raspberry Pi 5,“ Raspberry Pi Ltd, Techn. Ber., Apr. 2026. besucht am 7. Juni 2026.
- [19] Raspberry Pi Ltd, „Raspberry Pi 4 Model B Specifications,“ Raspberry Pi Ltd, Techn. Ber. besucht am 7. Juni 2026.
- [20] Raspberry Pi Ltd, „Raspberry Pi OS,“ Raspberry Pi Ltd, Techn. Ber. besucht am 7. Juni 2026.
- [21] Raspberry Pi Ltd, *Raspberry Pi OS Downloads*. besucht am 7. Juni 2026.
- [22] Saad Imtiaz, *Raspberry Pi 5 vs Raspberry Pi 4: A Comparison*, Nov. 2023. besucht am 7. Juni 2026.

- [23] E. Schubert, J. Sander, M. Ester, H.-P. Kriegel und X. Xu, „DBSCAN Revisited, Revisited: Why and How You Should (Still) Use DBSCAN,“ *ACM Transactions on Database Systems*, Jg. 42, Nr. 3, 19:1–19:21, 2017. DOI: 10.1145/3068335
- [24] Ubuntu Documentation, *System Requirements*, Website, Juli 2026. besucht am 7. Juni 2026.
- [25] F. Wilms, *Entscheidungen meistern: Eine Einführung in praxistaugliche Entscheidungsmethoden*. 2025. DOI: 10.1007/978-3-662-70764-7

A Anhang

```
1 #!/bin/bash
2 # =====
3 # Raspberry Pi 4 + Ubuntu Server
4 # Einrichtung eines WLAN-Access-Points mit hostapd und dnsmasq
5 #
6 # HINWEISE:
7 # - Dieses Skript dient als dokumentierte Installationsanleitung.
8 # - Einige Schritte erfordern manuelle Kontrolle der
   Konfigurationsdateien.
9 # - Vor Änderungen immer prüfen, wie die vorhandene Konfiguration
   aussieht.
10 # - Die Netplan-Datei kann anders heißen als 01-rpi.yaml.
11 # =====
12
13 sudo apt update && sudo apt upgrade -y
14
15 sudo apt install -y hostapd dnsmasq git gpod libgpod-dev
16
17 sudo reboot
18
19 sudo systemctl stop hostapd
20 sudo systemctl stop dnsmasq
21
```

```
22 # Netplan-Datei prüfen
23 ls -l /etc/netplan/
24 sudo nano /etc/netplan/01-rpi.yaml
25 # Inhalt:
26 #
27 # network:
28 #   version: 2
29 #   renderer: networkd
30 #
31 #   ethernets:
32 #     eth0:
33 #       dhcp4: true
34
35 sudo chmod 600 /etc/netplan/01-rpi.yaml
36 sudo netplan apply
37
38 # wlan0 mit statischer IP versehen
39 sudo nano /etc/systemd/network/08-wlan0.network
40 # Inhalt:
41 #
42 # [Match]
43 # Name=wlan0
44 #
45 # [Network]
46 # Address=192.168.4.1/24
47
48 sudo systemctl restart systemd-networkd
49
50 # dnsmasq konfigurieren
51 sudo mv /etc/dnsmasq.conf /etc/dnsmasq.conf.orig
52 sudo nano /etc/dnsmasq.conf
53 # Inhalt:
54 #
55 # interface=wlan0
56 # bind-interfaces
```

```

57 # dhcp-range=192.168.4.20,192.168.4.100,255.255.255.0,12h
58 # domain-needed
59 # bogus-priv
60
61 # hostapd konfigurieren
62 sudo nano /etc/hostapd/hostapd.conf
63 # Inhalt:
64 #
65 # interface=wlan0
66 # driver=nl80211
67 # ssid=raspberrypi
68 # hw_mode=g
69 # channel=7
70 #
71 # auth_algs=1
72 # wpa=2
73 # wpa_passphrase=raspberrypi
74 # wpa_key_mgmt=WPA-PSK
75 # rsn_pairwise=CCMP
76 #
77 # country_code=DE
78 # ieee80211n=1
79
80 sudo nano /etc/default/hostapd
81 # Inhalt:
82 #
83 # DAEMON_CONF="/etc/hostapd/hostapd.conf"
84
85 # Dienste aktivieren
86 sudo systemctl unmask hostapd
87 sudo systemctl enable hostapd dnsmasq
88
89 sudo systemctl start hostapd
90 sudo systemctl start dnsmasq
91

```

```
92 # Status prüfen
93 systemctl status hostapd
94 systemctl status dnsmasq
95
96 # GPIO-Controller prüfen
97 gpioinfo
98
99 # Erwartete Ergebnisse:
100 # - hostapd: active (running)
101 # - dnsmasq: active (running)
102 # - gpioinfo liefert die erwarteten GPIO-Chips
```

Listing 1: Einrichtung eines WLAN-Access-Points auf einem Raspberry Pi 4 mit Ubuntu Server